

UNIVERZITET U BEOGRADU
MATEMATIČKI FAKULTET



Dunja Čitlučanin

PRONALAZENJE POČETNOG REGIONA
REPLIKACIJE DNK - ELEKTRONSKA
LEKCIJA

master rad

Beograd, 2026.

Mentor:

dr Jovana KOVAČEVIĆ, vanredni profesor
Univerzitet u Beogradu, Matematički fakultet

Članovi komisije:

dr Mirjana MALJKOVIĆ RUŽIČIĆ, docent
Univerzitet u Beogradu, Matematički fakultet

Nevena ĆIRIĆ, asistent
Univerzitet u Beogradu, Matematički fakultet

Datum odbrane: _____

Posveta

Naslov master rada: Pronalaženje početnog regiona replikacije DNK - elektronska lekcija

Rezime: Ovaj rad predstavlja elektronsku lekciju posvećenu algoritmima za identifikaciju početnog regiona replikacije u DNK sekvencama. Lekcija obuhvata teorijsko objašnjenje osnovnih bioinformatičkih koncepata i algoritama, uključujući *Frequent Words*, *Clump Finding*, *GC-skew* analizu i pristupe sa dozvoljenim odstupanjima, uz analizu njihovog rada i vremenske složenosti. Interaktivni deo realizovan je kao veb aplikacija korišćenjem tehnologija *FastAPI* i *React*, koja omogućava pokretanje algoritama implementiranih u programskom jeziku *Python*. Korisnicima je omogućeno da eksperimentišu sa različitim ulaznim podacima i prate rezultate izvršavanja algoritama, čime se olakšava razumevanje njihovih principa rada i primene u bioinformatici. Ovakva elektronska lekcija može služiti kao edukativni alat za studente i nastavnike u oblasti bioinformatike.

Ključne reči: DNK, bioinformatika, algoritmi, replikacija, *oriC*, *Frequent Words*, *Clump Finding*, *GC-skew*, *mismatches*

Sadržaj

1	Uvod	1
2	Teorijske osnove	2
2.1	Struktura DNK	2
2.2	Replikacija DNK	3
2.3	Početni region replikacije (<i>oriC</i>)	4
2.4	DNK polimeraza, <i>DnkA</i> proteini i boksovi	4
2.5	Obrnuti komplement (<i>Reverse Complement</i>)	5
3	Algoritamski pristupi za pronalaženje početnog regiona replikacije	7
3.1	Problem čestih reči (<i>Frequent Words Problem</i>)	7
3.1.1	Algoritam	8
3.1.2	Vremenska složenost	10
3.1.3	Optimizovani pristup korišćenjem niza frekvencija	11
3.1.4	Optimizovani pristup korišćenjem heš mape	15
3.1.5	Pronalaženje najčešćih <i>k</i> -grama sortiranjem	18
3.1.6	Poređenje različitih pristupa	20
3.2	Problem pronalaženja grupa (<i>Clump Finding Problem</i>)	21
3.2.1	Algoritam	22
3.2.2	Vremenska složenost naivnog pristupa	23
3.2.3	Optimizacija kliznog prozora	25
3.2.4	Vremenska složenost optimizovanog pristupa	26
3.3	Asimetrija replikacije DNK i <i>Skew</i> dijagram	27
3.3.1	Okazaki fragmenti i asimetrija replikacije	28
3.3.2	Iskrivljeni dijagram (<i>Skew Diagram</i>)	31
3.3.3	Vremenska složenost	33
3.3.4	Prednosti i ograničenja	34

3.4	Približno poklapanje sekvenci i česte reči sa propustima	35
3.4.1	Hamingova distanca	35
3.4.2	Približno poklapanje obrazaca (<i>Approximate Pattern Matching</i>)	36
3.4.3	Prebrojavanje približnih pojavljivanja	38
3.4.4	Vremenska složenost približnog poklapanja	39
3.4.5	Problem čestih reči sa propustima (<i>Frequent Words with Mismatches</i>)	39
3.4.6	d -susedstvo obrasca	40
3.4.7	Algoritam za česte reči sa propustima	41
3.4.8	Uzimanje u obzir obrnutih komplementa	43
3.4.9	Vremenska složenost	43
3.4.10	Optimizovana implementacija za česte reči sa propustima . . .	45
3.5	Ograničenja i otvoreni problemi	46
3.5.1	Ograničenja frekvencijskih pristupa	46
3.5.2	Struktura DNK i dodatni biološki faktori	47
4	Elektronska lekcija	50
4.1	Arhitektura aplikacije	50
4.2	Korišćene tehnologije	51
4.3	Pokretanje aplikacije u lokalnom okruženju	51
4.3.1	Pokretanje serverske komponente	52
4.3.2	Pokretanje klijentske komponente	52
4.4	Postavljanje aplikacije na javno dostupne servise	53
4.4.1	Postavljanje serverske komponente na <i>Render</i>	53
4.4.2	Postavljanje klijentske komponente na <i>Vercel</i>	54
4.4.3	Povezivanje klijentske i serverske komponente	55
4.5	Funkcionalnosti elektronske lekcije	57
4.5.1	Početna stranica	57
4.5.2	Teorijske osnove	58
4.5.3	Česte reči	59
4.5.4	Pronalaženje grupa	60
4.5.5	<i>GC-skew</i> dijagram	61
4.5.6	Približno poklapanje i česte reči sa propustima	62
4.5.7	Literatura	64
5	Zaključak	65

Glava 1

Uvod

Dezoksiribonukleinska kiselina (DNK) je osnovni nosilac genetske informacije kod živih organizama. Jedan od značajnih izazova u bioinformatici predstavlja identifikacija funkcionalno relevantnih regiona unutar DNK sekvenci, pri čemu se ova analiza često zasniva na primeni algoritamskih metoda.

Posebno interesantan problem predstavlja određivanje početnog regiona replikacije (*oriC*), koji ima ključnu ulogu u procesu umnožavanja genetskog materijala. U okviru velikih genoma, koji mogu sadržati milione nukleotida, ovaj region je relativno kratak i nije eksplicitno označen. Stoga se njegovo identifikovanje zasniva na analizi obrazaca u DNK sekvenci i prepoznavanju karakterističnih signala koji ukazuju na početak replikacije.

Jedan od ključnih uvida jeste da se u ovim regionima često pojavljuju ponavljajuće sekvence, kao i specifične statističke nepravilnosti u raspodeli nukleotida. Ove osobine omogućavaju primenu algoritamskih metoda za identifikaciju kandidata za *oriC* region.

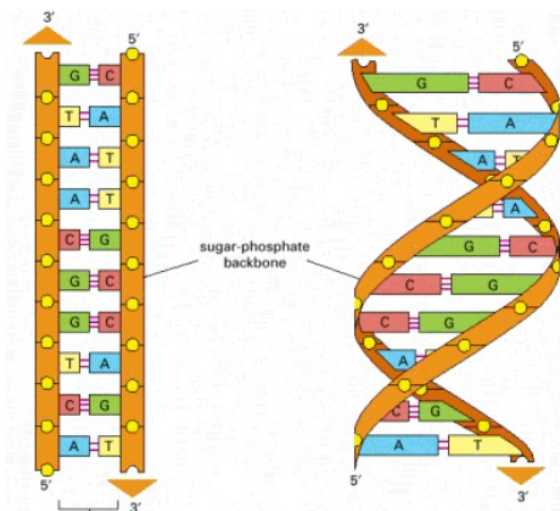
U ovom radu analizirani su algoritamski pristupi koji se zasnivaju na učestalosti podniski, njihovoj lokalnoj raspodeli i statističkim svojstvima genoma. Cilj rada je da se prikaže kako se kombinacijom ovih metoda može efikasno pristupiti problemu identifikacije početnog regiona replikacije. Rad je organizovan na sledeći način. U drugom poglavlju predstavljene su osnovne biološke i bioinformatičke osnove koje su neophodne za razumevanje problema. U trećem poglavlju obrađeni su algoritamski pristupi za pronalaženje početnog regiona replikacije. U četvrtom poglavlju prikazana je implementacija elektronske lekcije i tehnologije korišćene za realizaciju interaktivnog dela rada.

Glava 2

Teorijske osnove

2.1 Struktura DNK

DNK je molekul koji se sastoji od dva međusobno povezana lanca koji formiraju dvostruku spiralu. Svaki lanac je građen od nukleotida, pri čemu svaki nukleotid sadrži jednu od četiri azotne baze: adenin (A), timin (T), guanin (G) ili citozin (C). Kao što je prikazano na Slici 2.1, baze se međusobno uparuju po principu komplementarnosti, tako da se adenin vezuje za timin, a guanin za citozin. Ovakva struktura omogućava stabilnost molekula, kao i precizno kopiranje genetske informacije tokom procesa replikacije [10].



Slika 2.1: Struktura DNK [10]

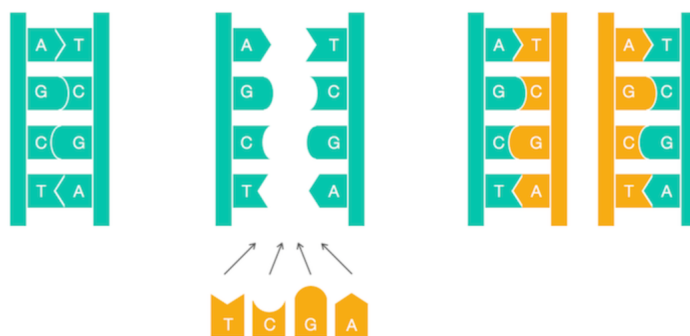
DNK je u ćelijama organizovana u strukture koje se nazivaju hromozomi. Skup svih DNK molekula jednog organizma, odnosno kompletna informacija sadržana u

njegovim hromozomima, naziva se genom. Različiti organizmi imaju različit broj hromozoma, kao i genom različite dužine, koji može varirati od nekoliko stotina hiljada do više miliona nukleotida.

Sa stanovišta bioinformatike, DNK sekvenca se može predstaviti kao konačna niska karaktera nad alfabetom $\{A, C, G, T\}$. Podniske te niske, dužine k , nazivaju se k -grami i predstavljaju osnovne jedinice koje se analiziraju u bioinformatici. Analiza učestalosti i raspodele k -grama u genomu omogućava otkrivanje obrazaca koji mogu ukazivati na funkcionalno značajne regione DNK. Ovakav model omogućava primenu algoritamskih metoda za analizu sekvenci, prepoznavanje obrazaca i rešavanje različitih problema u obradi genetskih podataka.

2.2 Replikacija DNK

Replikacija DNK predstavlja proces umnožavanja genetskog materijala koji se odvija pre deobe ćelije. Tokom ovog procesa, dva lanca DNK se razdvajaju, a zatim se za svaki od njih sintetiše novi komplementarni lanac. Kao rezultat replikacije nastaju dve identične kopije DNK molekula, koje se raspodeljuju u ćerke ćelije. Slika 2.2 ilustruje opisano.

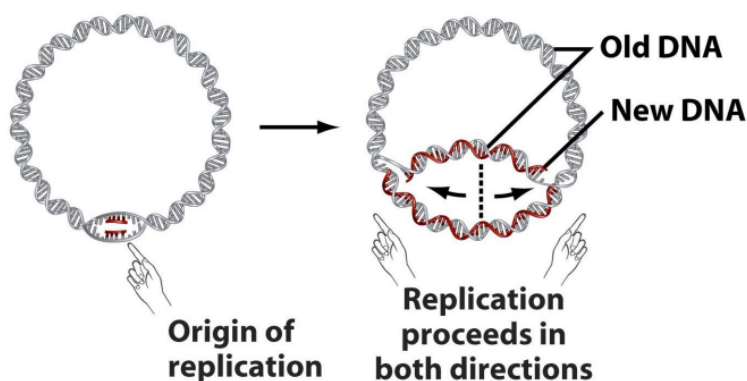


Slika 2.2: Replikacija DNK[14]

Ovaj proces je visoko kontrolisan i započinje na određenim lokacijama u genomu. Replikacija se zatim odvija u oba smera duž DNK lanca, pri čemu različiti enzimi i proteini učestvuju u razdvajanju lanaca i sintezi novih nukleotida.

2.3 Početni region replikacije (*oriC*)

Početni region replikacije, označen kao *oriC*, predstavlja specifičan segment DNK u kojem započinje proces replikacije. Kod bakterija, genom je najčešće organizovan u obliku kružnog hromozoma i sadrži samo jedan takav region iz kojeg se replikacija širi u oba smera, kao što je prikazano na Slici 2.3.



Slika 2.3: Početni region replikacije [12]

Ovaj region sadrži sekvence koje omogućavaju vezivanje proteina neophodnih za početak replikacije i time ima ključnu ulogu u pokretanju celokupnog procesa umnožavanja genetskog materijala. Iako je *oriC* relativno kratak u poređenju sa dužinom celog genoma, koji može sadržati milione nukleotida, on u DNK sekvenci nije eksplicitno označen. Zbog toga, njegovo pronalaženje predstavlja složen i netrivialan zadatak.

2.4 DNK polimeraza, *DnkA* proteini i boksovi

Početak replikacije DNK započinje vezivanjem specifičnih proteina za određene delove genoma. Jedan od najvažnijih proteina u ovom procesu je *DnkA*, koji ima sposobnost da prepozna i veže se za kratke sekvence unutar početnog regiona replikacije. Ove kratke sekvence poznate su kao *DnkA* boksovi i karakteriše ih to što se pojavljuju više puta unutar *oriC* regiona. Njihova učestalost na ograničenom regionu unutar DNK omogućava proteinima da identifikuju mesto na kojem treba da započne replikacija. Nakon vezivanja *DnkA* proteina dolazi do lokalnog razdvajanja DNK lanca, čime se omogućava delovanje *DNK polimeraze*, enzima koji funkcioniše kao molekularna mašina za kopiranje genetskog materijala i vrši sintezu novih DNK

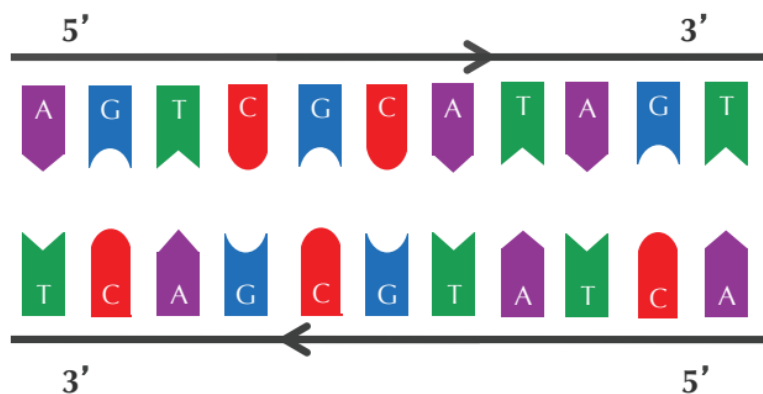
lanaca. Upravo zbog svoje ponavljajuće prirode, *DnkA* boksovi se mogu posmatrati kao karakteristični obrasci u DNK sekvenci, što omogućava njihovo pronalaženje primenom algoritamskih metoda.

2.5 Obrnuti komplement (*Reverse Complement*)

Zbog komplementarne prirode DNK molekula, svaki lanac DNK povezan je sa drugim lancem prema pravilima uparivanja baza, pri čemu se adenin (A) uparuje sa timinom (T), dok se guanin (G) uparuje sa citozinom (C). Pored toga, dva lanca DNK molekula imaju suprotne smerove prostiranja, odnosno jedan je orijentisan u smeru $5' \rightarrow 3'$, a drugi u smeru $3' \rightarrow 5'$.

Na slici 2.4 prikazana su dva komplementarna DNK lanca i način uparivanja nukleotida. Može se uočiti da svakoj bazi jednog lanca odgovara tačno određena komplementarna baza na drugom lancu.

Na osnovu ovih osobina definiše se pojam obrnutog komplementa (*reverse complement*). Komplement jedne DNK sekvence dobija se zamenom svake baze njenom komplementarnom bazom. Zbog suprotne orijentacije DNK lanaca, komplementarna sekvenca čita se u smeru $5' \rightarrow 3'$, čime se dobija obrnuti komplement.



Slika 2.4: Komplementarni DNK lanci i suprotna orijentacija lanaca [14]

Na primer, za sekvencu AGTC, komplementarna sekvenca je TCAG, dok je njen obrnuti komplement GACT.

S obzirom na dvostruku strukturu DNK, biološki značajne sekvence mogu se pojaviti na oba lanca molekula. Zbog toga je pri analizi genoma neophodno razmatrati i DNK sekvencu i njen obrnuti komplement. U kontekstu algoritama za pronalaženje početnog regiona replikacije, uzimanje u obzir obrnutih komplementarnih sekvenci

ima poseban značaj, jer se karakteristični obrasci mogu pojaviti u oba smera duž DNK molekula.

Glava 3

Algoritamski pristupi za pronalaženje početnog regiona replikacije

Biološke osobine početnog regiona replikacije, kao što su prisustvo ponavljajućih sekvenci i statističke nepravilnosti u raspodeli nukleotida, omogućavaju primenu algoritamskih metoda za njegovo pronalaženje. Analizom učestalosti podniski, njihove lokalne raspodele i statističkih karakteristika genoma moguće je identifikovati potencijalne kandidate za *oriC* region. U ovom poglavlju biće predstavljeni algoritamski pristupi koji koriste ove osobine za analizu DNK sekvenci.

3.1 Problem čestih reči (*Frequent Words Problem*)

Kako određeni obrasci u DNK sekvenci mogu ukazivati na funkcionalno važne regione, prirodan pristup jeste analiza učestalosti pojavljivanja podniski. Sekvence koje se pojavljuju značajno češće od ostalih mogu imati posebnu biološku ulogu, pa se problem svodi na pronalaženje najčešće zastupljenih podniski određene dužine.

Polazeći od pretpostavke da se DNK sekvenca može posmatrati kao niska karaktera nad alfabetom $\{A, C, G, T\}$, moguće je primeniti metode analize teksta u cilju identifikacije karakterističnih obrazaca.

Podniske dužine k jedne biološke sekvence (DNK, njenog segmenta ili proteina) nazivaju se k -grami ili k -meri i predstavljaju osnovne jedinice analize bioloških se-

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAZENJE POČETNOG REGIONA REPLIKACIJE

kvenci. Analizom frekvencije njihovog pojavljivanja moguće je identifikovati obrasce koji mogu ukazivati na prisustvo funkcionalno značajnih regiona, kao što je *oriC*.

Neka je data niska *Text* i podniska *Pattern* dužine k . Definišemo funkciju:

$$PatternCount(Text, Pattern)$$

kao broj pojavljivanja podniske *Pattern* u niski *Text*, pri čemu se uzimaju u obzir i preklapajuća pojavljivanja.

Algoritam za prebrojavanje pojavljivanja

Za izračunavanje funkcije $PatternCount(Text, Pattern)$ koristi se pristup zasnovan na kliznom prozoru (*sliding window*). Ideja je da se prolaskom kroz tekst proveri svaka podniska dužine k .

Algoritam 1 prikazuje pseudokod za izračunavanje ove funkcije.

Algoritam 1: Pattern Count

Funkcija $PatternCount(Text, Pattern)$

```
    count  $\leftarrow$  0
    for  $i \leftarrow 0$  to  $|Text| - |Pattern|$  do
        if  $Text[i..i + |Pattern| - 1] = Pattern$  then
            count  $\leftarrow$  count + 1
        end
    end
    return count
```

3.1.1 Algoritam

Problem čestih reči može se formalno definisati na sledeći način:

- Ulaz: niska karaktera *Text* i ceo broj k
- Izlaz: skup svih k -grama koji maksimizuju $PatternCount(Text, Pattern)$

Kažemo da je k -gram najčešći ukoliko se nijedan drugi k -gram ne pojavljuje više puta od njega u posmatranoj sekvenci.

Ideja i opis algoritma

Osnovna ideja algoritma za pronalazak čestih reči zasniva se na prolasku kroz celu DNK sekvencu i prebrojavanju koliko puta se svaka podniska dužine k pojavljuje. Nakon toga se izdvajaju one podniske koje se pojavljuju maksimalan broj puta.

Naivni pristup rešavanju ovog problema može se opisati kroz sledeće korake:

- Proći kroz sve moguće pozicije u niski *Text*
- Za svaku poziciju izdvojiti podnisku dužine k
- Za svaku podnisku izračunati $PatternCount(Text, Pattern)$ primenom algoritma 1
- Odrediti maksimalan broj pojavljivanja
- Izdvojiti sve podniske koje imaju tu frekvenciju

Algoritam 2 prikazuje pseudokod algoritma za pronalaženje najčešćih reči.

Algoritam 2: Frequent Words

```
Funkcija FrequentWords(Text,  $k$ )  
    najcesciKgrami  $\leftarrow$  prazan skup  
    for  $i \leftarrow 0$  to  $|Text| - k$  do  
        |  $pattern \leftarrow Text[i..i + k - 1]$   
        |  $count[i] \leftarrow PatternCount(Text, pattern)$   
    end  
     $maxBroj \leftarrow$  maksimalna vrednost u nizu count  
    for  $i \leftarrow 0$  to  $|Text| - k$  do  
        | if  $count[i] = maxBroj$  then  
        |     | dodaj  $Text[i..i + k - 1]$  u najcesciKgrami  
        | end  
    end  
    ukloni duplikate iz najcesciKgrami  
    return najcesciKgrami
```

Primer

Posmatrajmo DNK sekvencu:

ACTGACTCCCACCCC

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAZENJE POČETNOG REGIONA REPLIKACIJE

Za $k = 3$ izdvajaju se svi 3-grami i računa se broj njihovih pojavljivanja u sekvenci [14].

Neki od izdvojenih 3-grama su:

ACT, CTG, TGA, GAC, ACT, ...

Algoritam **PatternCount** za svaki obrazac prolazi kroz celu sekvencu i određuje koliko puta se dati 3-gram pojavljuje. Važno je napomenuti da algoritam **PatternCount** uzima u obzir i preklapajuća pojavljivanja podniski. To znači da nova pojava podniske može započeti pre nego što se prethodna potpuno završi.

Na primer, u nizu:

ATATATA

podniska ATA pojavljuje se na pozicijama 0, 2 i 4, pri čemu se ova pojavljivanja međusobno preklapaju.

Primenom algoritma **PatternCount** za svaki izdvojeni 3-gram računa se broj njegovih pojavljivanja u celoj sekvenci. Na slici 3.1 prikazan je niz **Count** kreiran primenom algoritma 2.

Text	A	C	T	G	A	C	T	C	C	C	A	C	C	C	C
COUNT	2	1	1	1	2	1	1	3	1	1	1	3	3		

Slika 3.1: Primer izdvajanja 3-grama i određivanja njihovih frekvencija [14].

Na primer:

$$\text{PatternCount}(\text{Text}, \text{ACT}) = 2$$

jer se obrazac ACT pojavljuje dva puta u sekvenci.

Slično tome:

$$\text{PatternCount}(\text{Text}, \text{CCC}) = 3$$

što znači da je CCC najčešći 3-gram u datoj sekvenci.

3.1.2 Vremenska složenost

Algoritam **PatternCount** prolazi kroz sve moguće pozicije u niski *Text*, kojih ima $|Text| - k + 1$. Za svaku poziciju vrši se poređenje podniske dužine k , što zahteva do k poređenja karaktera. Zbog toga je vremenska složenost ovog algoritma:

$$O((|Text| - k + 1) \cdot k)$$

što se za velike vrednosti $|Text|$ može aproksimirati kao:

$$O(|Text| \cdot k)$$

Algoritam ***Frequent Words*** generiše $|Text| - k + 1$ k -grama i za svaki od njih poziva funkciju ***PatternCount***. Stoga je ukupan broj operacija:

$$(|Text| - k + 1) \cdot ((|Text| - k + 1) \cdot k)$$

što daje ukupnu vremensku složenost:

$$O((|Text| - k + 1)^2 \cdot k)$$

Za velike vrednosti ulaza, ova složenost se može pojednostaviti na:

$$O(|Text|^2 \cdot k)$$

Drugim rečima, algoritam ima kvadratnu složenost u odnosu na dužinu sekvence, uz dodatni faktor k zbog poređenja podniski.

Ovakva složenost brzo postaje nepraktična za duge DNK sekvence, koje mogu sadržati milione nukleotida. Zbog toga je neophodno razmotriti optimizovane pristupe, poput korišćenja niza frekvencija ili sortiranja, koji omogućavaju značajno efikasnije rešavanje ovog problema.

3.1.3 Optimizovani pristup korišćenjem niza frekvencija

Naivni algoritam za rešavanje problema čestih reči ima visoku vremensku složenost jer za svaki k -gram ponovo prolazi kroz celu sekvencu. Da bi se ovaj problem rešio efikasnije, uvodi se optimizovani pristup zasnovan na korišćenju niza frekvencija.

Osnovna ideja je da se kroz sekvencu prolazi samo jednom, pri čemu se za svaki k -gram beleži broj njegovih pojavljivanja. Umesto ponovnog prebrojavanja, rezultati se čuvaju u strukturi podataka koja omogućava brz pristup.

Leksikografsko indeksiranje i numerička reprezentacija k -grama

Kako bi se omogućilo efikasno skladištenje i pretraživanje, svaki k -gram se mapira na jedinstveni indeks. S obzirom na to da DNK alfabet sadrži četiri simbola, postoji ukupno 4^k mogućih k -grama.

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAZENJE POČETNOG REGIONA REPLIKACIJE

Ovi k -grami se mogu poređati leksikografski (kao u rečniku), pri čemu se svakom dodeljuje jedinstveni broj iz opsega $[0, 4^k - 1]$.

Za ovu transformaciju koriste se funkcije ***PatternToNumber*** i ***NumberToPattern***, koje omogućavaju prelazak između simboličkog i numeričkog prikaza k -grama.

DNK alfabet $\{A, C, G, T\}$ može se predstaviti skupom brojeva:

$$A \rightarrow 0, \quad C \rightarrow 1, \quad G \rightarrow 2, \quad T \rightarrow 3$$

Funkcija ***SymbolToNumber*** predstavlja pomoćnu funkciju koja svakom simbolu DNK alfabeta (A, C, G, T) dodeljuje odgovarajuću numeričku vrednost (0, 1, 2, 3), čime omogućava numeričku reprezentaciju k -grama u sistemu sa osnovom 4.

Funkcija ***PatternToNumber*** preslikava dati k -gram u odgovarajući indeks tako što njegov prefiks tretira kao već izračunatu vrednost, koju zatim množi sa 4 i dodaje vrednost poslednjeg simbola. Ovaj postupak odgovara standardnoj konverziji iz pozicionog brojanog sistema i omogućava da se svaki k -gram jednoznačno predstavi celobrojnim indeksom. Algoritam 3 prikazuje pseudokod ove funkcije.

Algoritam 3: PatternToNumber

Funkcija PatternToNumber(*Pattern*)

if *Pattern* je prazan **then**
 return 0

end

symbol $\leftarrow$ poslednji simbol u *Pattern*

prefix $\leftarrow$ *Pattern* bez poslednjeg simbola

return $4 \cdot \text{PatternToNumber}(\text{prefix}) + \text{SymbolToNumber}(\text{symbol})$

Obrnuta transformacija vrši se funkcijom ***NumberToPattern***, koja dati indeks pretvara nazad u odgovarajući k -gram. Funkcija ***NumberToSymbol*** vrši obrnutu transformaciju u odnosu na funkciju ***SymbolToNumber***, tako što svakoj numeričkoj vrednosti iz skupa $\{0, 1, 2, 3\}$ dodeljuje odgovarajući simbol DNK alfabeta (A, C, G, T). Ova funkcija, Algoritam 4 prikazuje pseudokod, koristi operacije celobrojnog deljenja i ostatka pri deljenju sa 4: količnik određuje prefiks, dok ostatak određuje poslednji simbol. Na taj način se postupno rekonstruiše ceo k -gram.

Ove dve funkcije zajedno omogućavaju efikasan prelazak između simboličkog i numeričkog prikaza k -grama, što je ključno za implementaciju algoritama zasnovanih na nizovima frekvencija.

Algoritam 4: NumberToPattern

```

Funkcija NumberToPattern(index, k)
    if k = 1 then
        | return NumberToSymbol(index)
    end
    prefixIndex  $\leftarrow \lfloor \text{index}/4 \rfloor$ 
    r  $\leftarrow \text{index} \bmod 4$ 
    symbol  $\leftarrow$  NumberToSymbol(r)
    prefixPattern  $\leftarrow$  NumberToPattern(prefixIndex, k - 1)
    return konkatencija prefixPattern i symbol

```

Algoritam

Algoritam za izračunavanje frekvencija prolazi kroz tekst samo jednom i za svaki k -gram povećava odgovarajući element niza frekvencija. Pseudokod prikazuje Algoritam 5.

Algoritam 5: Computing Frequencies

```

Funkcija ComputingFrequencies(Text, k)
    for i  $\leftarrow$  0 to  $4^k - 1$  do
        | FrequencyArray[i]  $\leftarrow$  0
    end
    for i  $\leftarrow$  0 to |Text| - k do
        | Pattern  $\leftarrow$  Text[i..i + k - 1]
        | j  $\leftarrow$  PatternToNumber(Pattern)
        | FrequencyArray[j]  $\leftarrow$  FrequencyArray[j] + 1
    end
    return FrequencyArray

```

Nakon što je niz frekvencija izračunat, najčešći k -grami se dobijaju pronalazanjem maksimalne vrednosti u nizu i identifikacijom svih indeksa koji imaju tu vrednost. Algoritam 6 prikazuje pseudokod funkcije *Faster Frequent Words*.

Primer

Za slučaj $k = 2$ postoji ukupno $4^2 = 16$ mogućih 2-grama. Oni se mogu poredati leksikografski, pri čemu svaki 2-gram dobija jedinstveni indeks iz opsega od 0 do 15.

Na slici 3.2 prikazan je primer leksikografskog uređenja svih mogućih 2-grama, zajedno sa odgovarajućim indeksima i nizom frekvencija za posmatranu DNK sekvencu.

Algoritam 6: Faster Frequent Words

```

Funkcija FasterFrequentWords(Text, k)
    najcesciKgrami  $\leftarrow$  prazan skup
    FrequencyArray  $\leftarrow$  ComputingFrequencies(Text, k)
    maxCount  $\leftarrow$  maksimalna vrednost u FrequencyArray
    for i  $\leftarrow$  0 to  $4^k - 1$  do
        if FrequencyArray[i] = maxCount then
            Pattern  $\leftarrow$  NumberToPattern(i, k)
            dodaj Pattern u skup rezultata
        end
    end
    return najcesciKgrami

```

Na primer, 2-gram GT ima indeks 11 što znači da funkcija: $PatternToNumber(GT) = 11$. Obrnuto, funkcija: $NumberToPattern(11, 2)$ vraća GT.

Na ovaj način omogućava se efikasno mapiranje između simboličkog i numeričkog prikaza k -grama, što predstavlja osnovu za konstrukciju niza frekvencija.

Niz frekvencija prikazan na Slici 3.2 predstavlja broj pojavljivanja svakog 2-grama u posmatranoj sekvenci. Na primer, vrednost 3 na poziciji koja odgovara 2-gramu AA pokazuje da se ovaj obrazac pojavljuje tri puta u sekvenci. Slično tome, 2-gram GG takođe ima frekvenciju 3, dok se neki obrasci ne pojavljuju nijednom, pa je njihova frekvencija jednaka 0. Na ovaj način niz frekvencija omogućava efikasno određivanje najčešćih k -grama bez ponovnog prolaska kroz celu sekvencu.

<i>k</i> -mer	AA	AC	AG	AT	CA	CC	CG	CT	GA	GC	GG	GT	TA	TC	TG	TT
index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
frequency	3	0	2	0	1	0	0	0	0	1	3	1	0	0	1	0

Slika 3.2: Leksikografsko indeksiranje 2-grama i primer niza frekvencija [14].

Vremenska složenost optimizovanog pristupa

U ovom pristupu tekst se obrađuje samo jednom, pri čemu se za svaki k -gram vrši njegova obrada i konverzija u numerički oblik, što daje složenost $O(|Text| \cdot k)$.

Međutim, dodatni trošak predstavlja inicijalizacija niza dimenzije 4^k , kao i iteracija kroz ceo niz prilikom traženja maksimuma i rekonstrukcije odgovarajućih k -grama. Ukupna složenost algoritma iznosi:

$$O(|Text| \cdot k + 4^k \cdot k)$$

Za male vrednosti k , izraz 4^k može se smatrati konstantom, pa složenost postaje približno linearna u odnosu na dužinu teksta.

Prostorna složenost ovog pristupa određena je dimenzijom niza frekvencija, koji sadrži 4^k elemenata. Zbog toga prostorna složenost iznosi:

$$O(4^k)$$

Ovaj pristup je značajno efikasniji od naivnog algoritma kada je k relativno malo u odnosu na $|Text|$. Međutim, zbog eksponencijalnog rasta izraza 4^k , ovaj metod postaje nepraktičan za veće vrednosti k , što motiviše razvoj dodatnih optimizacija.

3.1.4 Optimizovani pristup korišćenjem heš mape

Prethodno opisani pristup korišćenjem niza frekvencija omogućava efikasnije rešavanje problema čestih reči u odnosu na naivni algoritam, ali ima jedan važan nedostatak. Dimenzija niza frekvencija iznosi 4^k , jer se unapred rezerviše mesto za svaki mogući k -gram nad DNK alfabetom. Kada je vrednost k veća, broj mogućih k -grama raste eksponencijalno, pa ovaj pristup može zahtevati veliku količinu memorije.

Da bi se ovaj problem ublažio, uvodi se optimizovani pristup zasnovan na korišćenju heš mape, odnosno rečnika. Umesto da se čuva frekvencija za svaki mogući k -gram, u memoriji se čuvaju samo oni k -grami koji se zaista pojavljuju u posmatranoj DNK sekvenci.

Osnovna ideja je da se kroz tekst prolazi samo jednom. U svakom koraku izdvaja se trenutni k -gram, a zatim se u heš mapi ažurira broj njegovih pojavljivanja. Ključ u heš mapi predstavlja k -gram, dok vrednost predstavlja broj njegovih pojavljivanja u tekstu.

Ako se posmatrani k -gram prvi put pojavljuje, on se dodaje u heš mapu sa vrednošću 1. Ukoliko već postoji u heš mapi, njegova vrednost se uvećava za 1. Na taj način se izbegava ponovno prebrojavanje pojavljivanja istog obrasca, kao i čuvanje frekvencija za obrasce koji se uopšte ne pojavljuju u sekvenci.

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAZENJE POČETNOG REGIONA REPLIKACIJE

Algoritam

Algoritam za izračunavanje frekvencija pomoću heš mape prolazi kroz tekst i za svaki pronađeni k -gram ažurira odgovarajuću vrednost u rečniku. Pseudokod je prikazan u Algoritmu 7.

Algoritam 7: Computing Frequencies With Dictionary

```
Funkcija ComputingFrequenciesWithDictionary(Text, k)
    FrequencyMap  $\leftarrow$  prazna heš mapa
    for  $i \leftarrow 0$  to  $|Text| - k$  do
        Pattern  $\leftarrow Text[i..i + k - 1]$ 
        if Pattern nije ključ u FrequencyMap then
            FrequencyMap[Pattern]  $\leftarrow 1$ 
        end
        else
            FrequencyMap[Pattern]  $\leftarrow$  FrequencyMap[Pattern] + 1
        end
    end
    return FrequencyMap
```

Nakon što su frekvencije svih pronađenih k -grama izračunate, najčešći k -grami se dobijaju pronalazanjem maksimalne vrednosti u heš mapi. Kao što je prikazano u Algoritmu 8, svi obrasci čija je vrednost jednaka maksimumu dodaju se u skup rezultata [15].

Algoritam 8: Faster Frequent Words With Dictionary

```
Funkcija FasterFrequentWordsWithDictionary(Text, k)
    FrequentPatterns  $\leftarrow$  prazan skup
    FrequencyMap  $\leftarrow$  ComputingFrequenciesWithDictionary(Text, k)
    maxCount  $\leftarrow$  maksimalna vrednost u FrequencyMap
    foreach Pattern u FrequencyMap do
        if FrequencyMap[Pattern] = maxCount then
            dodaj Pattern u FrequentPatterns
        end
    end
    return FrequentPatterns
```

Primer

Neka je data DNK sekvenca ACTGACTCCCACCCC i neka je $k = 3$. Algoritam posmatra sve 3-grame koji se pojavljuju u tekstu pomeranjem prozora dužine 3 za

jedno mesto udesno.

Na početku je heš mapa prazna. Prvi posmatrani 3-gram je **ACT**, pa se on dodaje u heš mapu sa vrednošću 1. Zatim se posmatra 3-gram **CTG**, koji se takođe dodaje sa vrednošću 1. Kada se kasnije ponovo pojavi 3-gram **ACT**, njegova vrednost u heš mapi se uvećava sa 1 na 2.

Na ovaj način heš mapa postepeno beleži broj pojavljivanja svakog 3-grama koji se zaista javlja u sekvenci. Na kraju obrade, najčešći 3-gram je onaj čija je vrednost u heš mapi najveća. Ako više različitih 3-grama ima istu maksimalnu vrednost, svi oni predstavljaju rezultat problema čestih reči.

Vremenska složenost optimizovanog pristupa korišćenjem heš mape

U ovom pristupu tekst se obrađuje samo jednom, pri čemu se u svakom koraku izdvaja jedan k -gram i ažurira njegova vrednost u heš mapi. Izdvajanje podniskove dužine k zahteva $O(k)$ operacija, dok je pristup i ažuriranje vrednosti u heš mapi u prosečnom slučaju $O(1)$.

S obzirom na to da se ova operacija izvršava za svih $|Text| - k + 1$ pozicija, ukupna složenost ove faze iznosi:

$$O(|Text| \cdot k)$$

Nakon toga, potrebno je pronaći maksimalnu vrednost u heš mapi, što zahteva prolazak kroz sve njene elemente. U najgorem slučaju, broj različitih k -grama je najviše $|Text| - k + 1$, pa je za ovaj korak potrebno:

$$O(|Text|)$$

Kombinovanjem svih koraka dolazi se do ukupne vremenske složenosti:

$$O(|Text| \cdot k)$$

Za razliku od pristupa sa nizom frekvencija, ovde nije potrebno inicijalizovati niz dimenzije 4^k , niti prolaziti kroz sve moguće k -game. Pretražuju se samo oni obrasci koji su se stvarno pojavili u tekstu.

Prostorna složenost zavisi od broja različitih k -grama u posmatranoj sekvenci. U najgorem slučaju, svi izdvojeni k -grami mogu biti različiti, pa je broj elemenata u heš mapi najviše $|Text| - k + 1$. Kako svaki k -gram zauzima $O(k)$ memorije, ukupna prostorna složenost iznosi:

$$O(|Text| \cdot k)$$

Ovaj pristup je posebno koristan za veće vrednosti k , jer izbegava eksponencijalni memorijski trošak koji nastaje kod niza frekvencija. Zbog toga predstavlja praktičniju optimizaciju kada nije potrebno čuvati informacije o k -gramima koji se ne pojavljuju u ulaznoj sekvenci.

3.1.5 Pronalaženje najčešćih k -grama sortiranjem

Pored pristupa zasnovanog na nizu frekvencija, problem čestih reči može se rešiti i korišćenjem sortiranja. Osnovna ideja ovog pristupa je da se svi k -grami najpre transformišu u numeričke vrednosti, nakon čega se sortiraju, čime se identični elementi grupišu jedan do drugog.

Neka je data niska *Text*. Za svaku poziciju i izdvajamo k -gram $Text[i..i+k-1]$ i transformišemo ga u odgovarajući broj korišćenjem funkcije ***PatternToNumber***. Na taj način dobijamo niz indeksa koji zatim sortiramo. Algoritam 9 prikazuje pseudokod ovog pristupa.

Nakon sortiranja, identični k -grami se pojavljuju u uzastopnim segmentima niza, što omogućava jednostavno prebrojavanje njihovih pojavljivanja.

Primer

Posmatrajmo DNK sekvencu:

AAGCAAAGGTGGG

Za $k = 2$ izdvajaju se svi 2-grami redom kojim se pojavljuju u sekvenci i transformišu se u odgovarajući numerički indeks korišćenjem funkcije ***PatternToNumber***. Na taj način dobija se niz prikazan na Slici 3.3.

2-mer	AA	AG	GC	CA	AA	AA	AG	GG	GT	TG	GG	GG
INDEX	0	2	9	4	0	0	2	10	11	14	10	10

Slika 3.3: Izdvajanje i transformacija 2-grama [14].

Nakon sortiranja identični indeksi se grupišu jedan do drugog (Slika 3.4).

Ovakav raspored omogućava jednostavno određivanje frekvencija, jer se najčešći k -grami pojavljuju kao najduži uzastopni segmenti identičnih vrednosti u sortiranom nizu.

Algoritam 9: Finding Frequent Words by Sorting

Funkcija FindingFrequentWordsBySorting(*Text*, *k*)

```

    FrequentPatterns ← prazan skup
    for i ← 0 to |Text| − k do
        Pattern ← Text[i..i + k − 1]
        Index[i] ← PatternToNumber(Pattern)
        Count[i] ← 1
    end
    SortedIndex ← Sort(Index)
    for i ← 1 to |Text| − k do
        if SortedIndex[i] = SortedIndex[i − 1] then
            Count[i] ← Count[i − 1] + 1
        end
    end
    maxCount ← maksimalna vrednost u Count
    for i ← 0 to |Text| − k do
        if Count[i] = maxCount then
            Pattern ← NumberToPattern(SortedIndex[i], k)
            dodaj Pattern u FrequentPatterns
        end
    end
    return FrequentPatterns

```

2-mer	AA	AA	AA	AG	AG	CA	GC	GG	GG	GG	GT	TG
SORTEDINDEX	0	0	0	2	2	4	9	10	10	10	11	14
COUNT	1	2	3	1	2	1	1	1	2	3	1	1

Slika 3.4: Primer sortiranja indeksa i grupisanja identičnih *k*-grama [14].

U ovom primeru indeks 0 odgovara 2-gramu AA, dok indeks 10 odgovara 2-gramu GG. Kako se oba indeksa pojavljuju tri puta, zaključuje se da su AA i GG najčešći 2-grami u datoj sekvenci.

Vremenska složenost pristupa zasnovanog na sortiranju

U ovom pristupu generiše se niz od $|Text| - k + 1$ elemenata, nakon čega se vrši sortiranje tog niza. Dominantna operacija je sortiranje, čija složenost iznosi:

$$O(|Text| \log |Text|)$$

Dodatne operacije, poput transformacije *k*-grama i prolaska kroz niz, imaju složenost $O(|Text| \cdot k)$.

Ukupna vremenska složenost algoritma je:

$$O(|Text| \log |Text| + |Text| \cdot k)$$

3.1.6 Poređenje različitih pristupa

Na osnovu poređenja prikazanog u Tabeli 3.1 može se zaključiti da izbor algoritma zavisi od vrednosti parametra k , dužine ulazne sekvence i raspoloživih memorijskih resursa. Pristup zasnovan na nizu frekvencija efikasan je za manje vrednosti parametra k , dok je za veće vrednosti pogodniji pristup zasnovan na heš mapi, jer ne zahteva memoriju veličine 4^k . Pristup zasnovan na sortiranju predstavlja kompromis između vremenske i prostorne složenosti, dok je naivni algoritam prvenstveno pogodan za ilustraciju problema i obradu kraćih sekvenci.

Pristup	Vremenska složenost	Prostorna složenost	Karakteristike
Naivni algoritam	$O(Text ^2 \cdot k)$	$O(Text)$	Jednostavna implementacija, ali veoma spor za duže sekvence.
Niz frekvencija	$O(Text \cdot k + 4^k \cdot k)$	$O(4^k)$	Efikasan za male vrednosti k , ali zahteva memoriju veličine 4^k .
Heš mapa	$O(Text \cdot k)$	$O(Text \cdot k)$	Čuvaju se samo k -grami koji se pojavljuju u sekvenci.
Sortiranje	$O(Text \log Text + Text \cdot k)$	$O(Text)$	Ne zahteva niz veličine 4^k , ali je sporiji od pristupa sa heš mapom zbog sortiranja.

Tabela 3.1: Poređenje različitih pristupa za rešavanje problema čestih reči

3.2 Problem pronalazanja grupa (*Clump Finding Problem*)

Sama učestalost podniski na nivou celog genoma nije uvek dovoljna za preciznu identifikaciju funkcionalno značajnih regiona. U praksi je često mnogo značajnije posmatrati lokalnu frekvenciju podniski unutar kraćih segmenata genoma. Na primer, određeni k -gram može se pojavljivati relativno često u celom genomu, ali ukoliko su njegova pojavljivanja grupisana u kratkom regionu, takav obrazac može ukazivati na biološki značajan signal. Ovakva pojava je posebno izražena kod *DnkA* boksova, koji se često pojavljuju u grupama unutar početnog regiona replikacije, odnosno *oriC* regiona. Zbog toga analiza lokalnih ponavljanja predstavlja prirodno proširenje problema čestih reči.

Motivacija i veza sa biološkim problemom

Nakon identifikacije čestih obrazaca u poznatim *oriC* regionima, postavlja se ključno pitanje: kako pronaći slične regione u genomu kada početni region replikacije nije unapred poznat? U takvoj situaciji nije dovoljno posmatrati samo globalno najčešće k -game, jer se biološki relevantan signal često ogleda u njihovom lokalnom grupisanju.

Analiza *oriC* regiona bakterije *Vibrio cholerae*, prikazanog na Slici 3.5, pokazuje da se određeni 9-grami pojavljuju više puta u tom regionu.

```
atcaatgatcaacgtaagcttctaagcatgatcaaggtgctcacacagtttatccacaacctgagtgg
atgacatcaagataggtcggttgatctccttcctctcgtactctcatgaccacggaagatgatcaag
agaggatgatttcttgccatcatcgcaatgaatacttgtagcttggtgcttccaattgacatcttcagc
gccatattgctgctggccaaggtgacggagcggttattacgaaagcatgatcatggctgtgttctgttt
atcttgcttttgactgagacttgtaggatagacggtttttcatcactgactagccaaagccttactct
gcctgacatcgaccgtaaattgataatgaatttacatgcttccgcgacgatttacctcttgatcatcg
atccgattgaagatcttcaattgttaattctcttgccctcgactcatagccatgatgagctcttgatca
tgtttccttaaccctctattttttacggaagaatgatcaagctgctgctcttgatcatcgtttc
```

Slika 3.5: Početni region replikacije bakterije *Vibrio cholerae* [12]

Konkretno, u tom regionu izdvajaju se sledeći obrasci: ATGATCAAG i CTTGATCAT (Slika 3.6). Ova dva obrasca predstavljaju međusobne reverzne komplemente, što znači da nose istu biološku informaciju posmatrano na komplementarnim DNK lancima.

Ono što ih čini posebno interesantnim jeste činjenica da se oba pojavljuju više puta i to u neposrednoj blizini, unutar istog kratkog regiona genoma. Važno je

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAZENJE POČETNOG REGIONA REPLIKACIJE

```
atcaatgatcaacgtaagcttctaagcATGATCAAGgtgctcacacagtttatccacaacctgagtg  
atgacatcaagataggtcggttgatctccttcctctcgtactctcatgaccacggaagATGATCAAG  
agaggatgatttcttgccatcatcgcaatgaatacttgtagcttggtgcttccaattgacatcttcagc  
gccatattgcgctggccaaggtgacggagcgggattacgaaagcatgatcatggctgtgttctgttt  
atcttggttttgactgagacttgtaggatagacgggttttcatcactgactagccaaagccttactct  
gcctgacatcgaccgtaaatgataatgaatttacatgcttccgcgacgatttacctCTTGATCATcg  
atccgattgaagatcttcaattgttaattctcttgctcgactcatagccatgatgagctCTTGATCA  
TgtttcccttaaccctctattttttacggaagaATGATCAAGctgctgctCTTGATCATcgtttc
```

Slika 3.6: Početni region replikacije bakterije *Vibrio cholerae* sa označenim 9-gramima koji se najčešće pojavljuju [12]

naglasiti da ovakva pojava nije verovatna u slučajnoj DNK sekvenci. Verovatnoća da se isti 9-mer pojavi više puta u kratkom segmentu genoma je veoma mala. Na primer, verovatnoća da se neki 9-mer pojavi tri ili više puta u slučajno generisanom DNK nizu dužine 500 iznosi približno 1/1300. Zbog toga se ovakvi obrasci smatraju statistički značajnim, a upravo takvi grupisani motivi predstavljaju *DnkA* boksove.

Važno je naglasiti da *DnkA* boksovi nisu univerzalni i da se njihov konkretan nukleotidni sastav razlikuje između različitih organizama. Na primer, analiza *oriC* regiona bakterije *Thermotoga petrophila*, prikazanog na Slici 3.7, pokazuje da se obrasci **ATGATCAAG** i **CTTGATCAT**, koji su karakteristični za *Vibrio cholerae*, u ovom genomu uopšte ne pojavljuju. Umesto toga, izdvajaju se drugačiji često zastupljeni 9-grami, kao što su **AACCTACCA**, **ACCTACCAC**, **CCTACCACC** i drugi slični motivi.

```
aactctatacctcctttttgtcgaatttggtgatttatagagaaaatcttattaactgaaactaa  
aatggtaggtttGGTGGTAGGttttgtgtacattttgtagtatctgatttttaattacataccgta  
tattgtattaaattgacgaacaattgcatggaattgaatatatgcaaaacaaaCCTACCACCaaac  
tctgtattgaccatttttaggacaacttcagGGTGGTAGGtttctgaagctctcatcaatagactat  
tttagtctttacaacaatattaccgttcagattcaagattctacaacgctgttttaattggcggtt  
gcagaaaacttaccacctaataatccagtatccaagccgatttcagagaaacctaccacttacctac  
cacttaCCTACCACCcgggtggtgaagttgcagacattattaaaaacctcatcagaagcttggtcaa  
aaatttcaatactcgaaaCCTACCACCtgctgccctattatttactactactaataatagcagta  
taattgatctgaaaagaggtggttaaaaa
```

Slika 3.7: Početni region replikacije bakterije *Thermotoga petrophila* [12]

Da bismo formalizovali problem i započeli sa njegovim rešavanjem, ključna informacija je da se ovi obrasci pojavljuju lokalno grupisano. Zbog toga se uvodi problem pronalazjenja grupa (*Clump Finding Problem*), čiji je cilj identifikacija k -grama koji se pojavljuju više puta unutar ograničenog regiona genoma.

3.2.1 Algoritam

Problem pronalazjenja grupa formalno se definiše na sledeći način:

- Ulaz: niska *Genome* i celi brojevi k , L i t

- Izlaz: svi različiti k -grami koji formiraju (L, t) -grupe u niski *Genome*

Kažemo da k -gram formira (L, t) -grupu ukoliko postoji podniska genoma dužine L u kojoj se posmatrani k -gram pojavljuje najmanje t puta.

Drugim rečima, traže se svi obrasci dužine k koji su lokalno učestali u makar jednom segmentu genoma dužine L .

Naivni pristup

Osnovna ideja algoritma zasniva se na korišćenju kliznog prozora dužine L koji se pomera duž cele sekvence *Genome*. Za svaki položaj prozora posmatra se odgovarajući segment genoma, određuju se frekvencije svih k -grama u njemu, a zatim proverava da li se neki od njih pojavljuje najmanje t puta.

Ako određeni k -gram zadovoljava ovaj uslov u makar jednom prozoru, smatra se da formira grupu i uključuje se u skup rezultata.

Najdirektniji pristup rešavanju ovog problema jeste da se za svaki prozor dužine L iznova izračuna niz frekvencija svih k -grama. Nakon toga se prolazi kroz sve moguće indekse od 0 do $4^k - 1$ i proverava da li odgovarajući k -gram ima frekvenciju veću ili jednaku t .

Algoritam 10 prikazuje ovu osnovnu varijantu.

Primer

Pretpostavimo da se duž genoma pomera prozor dužine $L = 500$ i da se u svakom takvom segmentu posmatraju 9-grami, odnosno da je $k = 9$. Ukoliko se određeni 9-gram pojavi najmanje $t = 3$ puta unutar nekog prozora, tada kažemo da on formira $(500, 3)$ -grupu.

Na primer, ukoliko se obrazac ATGATCAAG tri puta pojavi unutar jednog segmenta dužine 500 nukleotida, taj obrazac predstavlja kandidat za biološki značajnu sekvencu. Slično važi i za njegov reverzni komplement. Upravo ovakvi lokalno grupisani obrasci predstavljaju signal koji može ukazivati na prisustvo *oriC* regiona.

3.2.2 Vremenska složenost naivnog pristupa

Analiza vremenske složenosti zasniva se na broju prozora i operacija koje se izvršavaju u svakom od njih.

Algoritam 10: Clump Finding

Funkcija `ClumpFinding`(*Genome*, *k*, *L*, *t*)

```

    FrequentPatterns ← prazan skup
    Clump ← niz dužine  $4^k$  inicijalizovan na 0
    for i ← 0 to |Genome| − L do
        | Text ← Genome[i..i + L − 1]
        | FrequencyArray ← ComputingFrequencies(Text, k)
        | for index ← 0 to  $4^k - 1$  do
            | | if FrequencyArray[index] ≥ t then
            | | | Clump[index] ← 1
            | | end
        | end
    end
    for i ← 0 to  $4^k - 1$  do
        | if Clump[i] = 1 then
        | | Pattern ← NumberToPattern(i, k)
        | | dodaj Pattern u FrequentPatterns
        | end
    end
    return FrequentPatterns

```

Broj prozora dužine *L* u niski *Genome* jednak je:

$$|Genome| - L + 1$$

Za svaki prozor najpre se računa niz frekvencija. Funkcija ***ComputingFrequencies*** zahteva inicijalizaciju niza dimenzije 4^k , što ima složenost $O(4^k)$, a zatim prolazak kroz prozor dužine *L*, pri čemu se za svaki od približno *L* *k*-grama vrši transformacija pomoću funkcije ***PatternToNumber***. Kako ova transformacija obrađuje *k* simbola, njen trošak je $O(k)$, pa ukupno računanje niza frekvencija zahteva:

$$O(4^k + L \cdot k)$$

Nakon toga se prolazi kroz svih 4^k mogućih indeksa kako bi se proverilo da li je frekvencija odgovarajućeg *k*-grama bar *t*, što zahteva dodatnih:

$$O(4^k)$$

operacija. Kombinovanjem ovih koraka dobija se da je ukupna složenost jedne iteracije prozora:

$$O(4^k + L \cdot k)$$

Pošto se takva obrada vrši za svih $|Genome| - L + 1$ prozora, ukupna vremenska složenost naivnog algoritma iznosi:

$$O((|Genome| - L + 1) \cdot (4^k + L \cdot k))$$

Kako je uobičajeno da važi $L \ll |Genome|$, broj prozora $(|Genome| - L + 1)$ može se aproksimirati sa $|Genome|$, pa se vremenska složenost često zapisuje i kao

$$O(|Genome| \cdot (4^k + L \cdot k)).$$

Za velike genome ovaj pristup postaje veoma skup, jer se za svaki prozor iznova računa niz frekvencija bez korišćenja prethodnih rezultata. U tipičnom slučaju bakterijskih genoma, gde je $|Genome|$ veoma veliko, a k i L fiksirani, ova složenost je i dalje previsoka za efikasnu obradu.

3.2.3 Optimizacija kliznog prozora

Glavni nedostatak naivnog pristupa jeste to što se niz frekvencija pri svakom pomeranju prozora računa iz početka, iako se između dva uzastopna prozora menja veoma mali broj podataka.

Kada se prozor pomeri za jednu poziciju udesno:

- jedan k -gram izlazi iz prozora,
- jedan novi k -gram ulazi u prozor.

Sve ostale frekvencije ostaju nepromenjene. To znači da nije potrebno iznova računati ceo niz frekvencija, već je dovoljno ažurirati samo dve njegove vrednosti:

- smanjiti frekvenciju k -grama koji izlazi iz prozora,
- povećati frekvenciju k -grama koji ulazi u prozor.

Na taj način dobija se efikasniji algoritam koji koristi isti princip kliznog prozora, ali izbegava redundantna izračunavanja.

Optimizovani algoritam

Algoritam 11 prikazuje optimizovanu verziju pronalaženja grupa. U prvom koraku računa se niz frekvencija za početni prozor dužine L , a zatim se pri svakom pomeranju prozora ažuriraju samo vrednosti koje odgovaraju prvom i poslednjem k -gramu u novom položaju prozora.

Algoritam 11: Better Clump Finding

```

Funkcija BetterClumpFinding(Genome,  $k$ ,  $t$ ,  $L$ )
    FrequentPatterns  $\leftarrow$  prazan skup
    Clump  $\leftarrow$  niz dužine  $4^k$  inicijalizovan na 0
    Text  $\leftarrow$  Genome[0.. $L - 1$ ]
    FrequencyArray  $\leftarrow$  ComputingFrequencies(Text,  $k$ )
    for  $i \leftarrow 0$  to  $4^k - 1$  do
        if FrequencyArray[ $i$ ]  $\geq t$  then
            Clump[ $i$ ]  $\leftarrow$  1
        end
    end
    for  $i \leftarrow 1$  to |Genome| -  $L$  do
        FirstPattern  $\leftarrow$  Genome[ $i - 1..i + k - 2$ ]
        indexFirst  $\leftarrow$  PatternToNumber(FirstPattern)
        FrequencyArray[indexFirst]  $\leftarrow$  FrequencyArray[indexFirst] - 1
        LastPattern  $\leftarrow$  Genome[ $i + L - k..i + L - 1$ ]
        indexLast  $\leftarrow$  PatternToNumber(LastPattern)
        FrequencyArray[indexLast]  $\leftarrow$  FrequencyArray[indexLast] + 1
        if FrequencyArray[indexLast]  $\geq t$  then
            Clump[indexLast]  $\leftarrow$  1
        end
    end
    for  $i \leftarrow 0$  to  $4^k - 1$  do
        if Clump[ $i$ ] = 1 then
            Pattern  $\leftarrow$  NumberToPattern( $i$ ,  $k$ )
            dodaj Pattern u FrequentPatterns
        end
    end
    return FrequentPatterns

```

3.2.4 Vremenska složenost optimizovanog pristupa

Optimizovani algoritam najpre računa niz frekvencija za prvi prozor, što zahteva:

$$O(4^k + L \cdot k)$$

Nakon toga, za svaki naredni prozor vrše se dve transformacije pomoću funkcije **PatternToNumber** i dve izmene u nizu frekvencija. Kako svaka transformacija zahteva $O(k)$ vremena, ukupni trošak jedne iteracije iznosi $O(k)$.

S obzirom na to da postoji $|Genome| - L$ pomeranja prozora, ukupni trošak ažuriranja frekvencija iznosi:

$$O((|Genome| - L) \cdot k)$$

Na kraju se prolazi kroz niz *Clump* dužine 4^k kako bi se rekonstruisali svi k -grami koji zadovoljavaju uslov. Ovaj korak uključuje konverziju indeksa u k -gram pomoću funkcije **NumberToPattern**, koja zahteva $O(k)$ vremena, pa ukupna složenost ovog dela iznosi:

$$O(4^k \cdot k)$$

Kombinovanjem svih koraka dobija se ukupna vremenska složenost optimizovanog algoritma:

$$O(4^k \cdot k + L \cdot k + (|Genome| - L) \cdot k)$$

Pošto važi

$$L \cdot k + (|Genome| - L) \cdot k = |Genome| \cdot k,$$

ukupna složenost može se zapisati kao

$$O(4^k \cdot k + |Genome| \cdot k).$$

Ovaj rezultat predstavlja značajno poboljšanje u odnosu na naivni pristup, jer se niz frekvencija više ne generiše iz početka za svaki prozor, čime se eliminiše multiplikativni faktor L .

3.3 Asimetrija replikacije DNK i *Skew* dijagram

Proces replikacije DNK započinje u specifičnom regionu genoma poznatom kao *oriC*, odakle se replikacija odvija u oba smera duž hromozoma, sve do regiona ozna-

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAZENJE POČETNOG REGIONA REPLIKACIJE

čenog kao *terC*. Tokom ovog procesa formiraju se dve replikacione viljuške (*replication forks*), duž kojih se replikacija odvija u suprotnim smerovima.

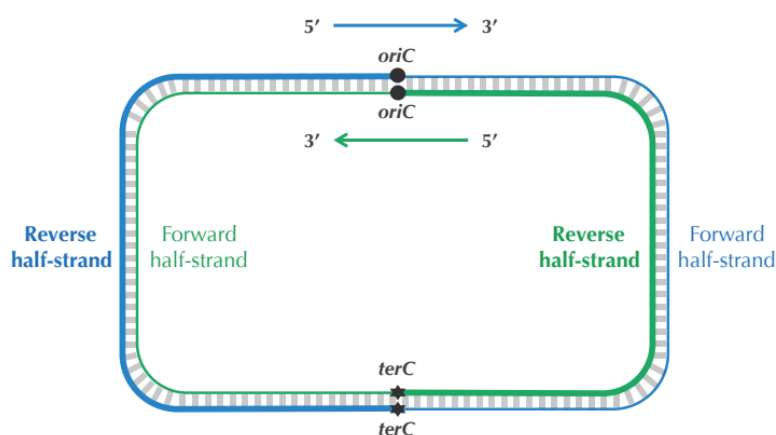
DNK polimeraza sintetiše novi lanac na osnovu postojećeg, ali može da dodaje nukleotide isključivo u smeru $5' \rightarrow 3'$. Postavlja se pitanje kako se vrši replikacija u suprotnom smeru, $3' \rightarrow 5'$. Ovo ograničenje dovodi do asimetrije u procesu replikacije i predstavlja osnovu za kasnije nastajanje razlika u raspodeli nukleotida duž genoma.

3.3.1 Okazaki fragmenti i asimetrija replikacije

Ograničenje da DNK polimeraza može da sintetiše novi lanac isključivo u smeru $5' \rightarrow 3'$ ima značajne posledice na proces replikacije. Iako se oba roditeljska lanca replikuju istovremeno, smer nastajanja duplikata nije isti.

Na osnovu smera replikacije, kružni bakterijski hromozom može se podeliti na dve polovine označene kao *forward half-strand* i *reverse half-strand*. Na *reverse half-strand* polovini hromozoma DNK polimeraza može kontinuirano da prati otvaranje DNK molekula i sintetiše novi lanac bez prekida. Nasuprot tome, na *forward half-strand* polovini hromozoma smer kretanja DNK polimeraze se razlikuje od orijentacije lanca, pa se sinteza ne može odvijati kontinualno.

Kao što je prikazano na Slici 3.8, hromozom se može podeliti na *reverse half-strand* polovinu i na *forward half-strand* polovinu odnosu na položaj početka replikacije (*oriC*).



Slika 3.8: Forward i reverse half-strand tokom procesa replikacije DNK [14].

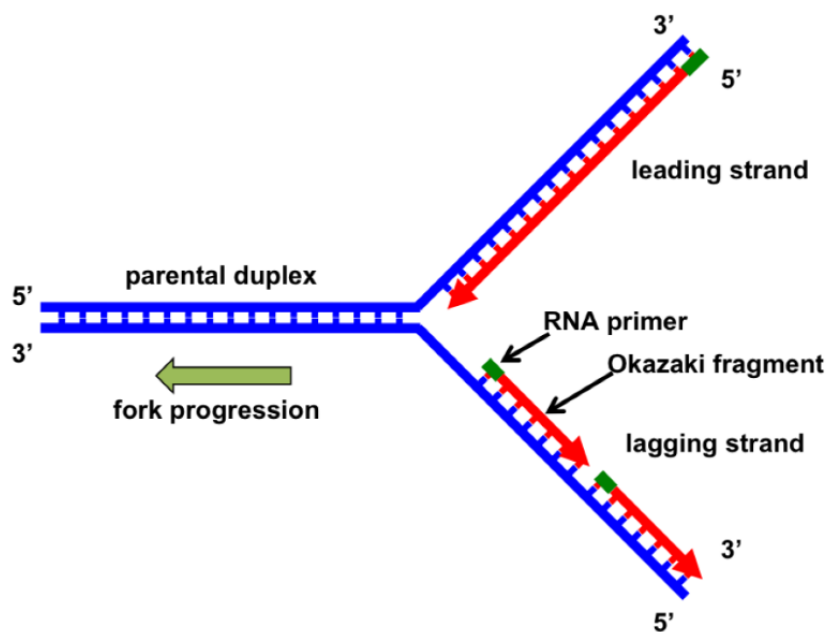
Pošto DNK polimeraza ne može da sintetiše DNK u smeru $3' \rightarrow 5'$, na *forward half-strand* polovini hromozoma neophodno je višestruko započinjanje sinteze. Svaki

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAZENJE POČETNOG REGIONA REPLIKACIJE

novi segment započinje formiranjem RNK prajmera, odnosno kratkog lanca RNK koji sintetiše enzim primaza i koji obezbeđuje početni 3' kraj sa koga DNK polimeraza može započeti sintezu. Ovi kratki DNK segmenti nazivaju se **Okazaki fragmenti**.

Kako se replikacioni viljuškasti region pomera, nastaju novi Okazaki fragmenti, a DNK polimeraza periodično započinje sintezu novih fragmenata. Po završetku replikacije, RNK prajmeri se uklanjaju, njihovo mesto popunjavaju DNK nukleotidi, a enzim DNK ligaza povezuje susedne fragmente u kontinuirani lanac. Zbog isprekidane sinteze, na *forward half-strand* polovini hromozoma potreban je poseban RNK prajmer za svaki novonastali Okazaki fragment. Nasuprot tome, na *reverse half-strand* polovini hromozoma sinteza se odvija kontinualno, pa je dovoljan samo jedan RNK prajmer [10].

Na Slici 3.9 prikazana je asimetrija replikacije DNK na *reverse* i *forward half-strand* polovini hromozoma, dok Slika 3.10 prikazuje ilustraciju koraka.

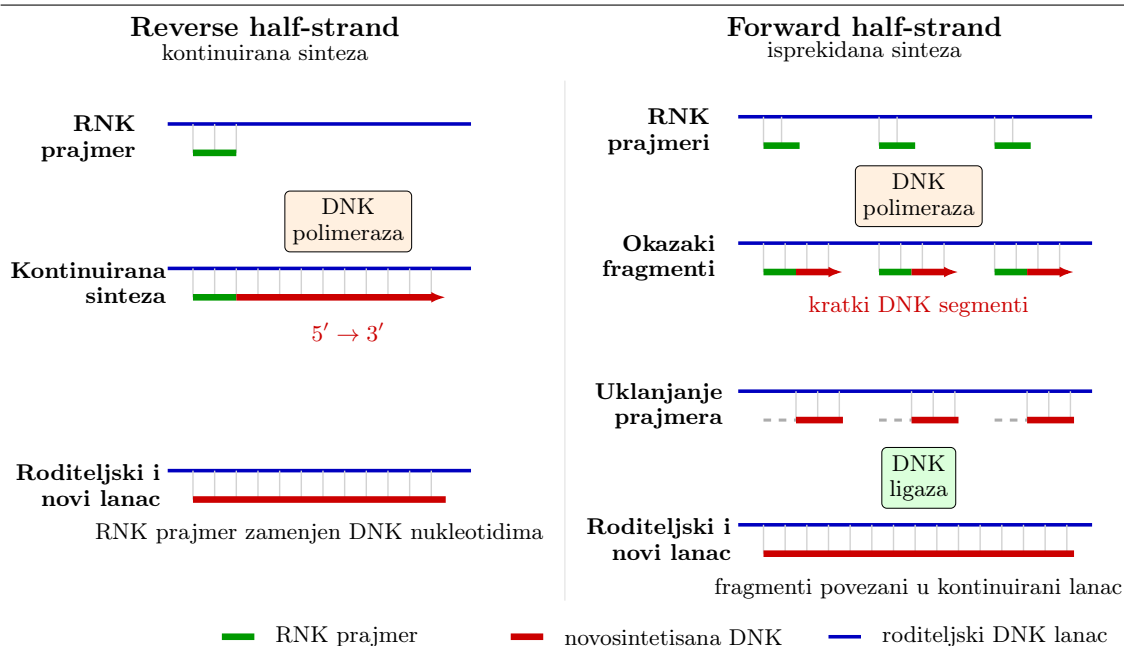


Slika 3.9: Asimetrija replikacije na *reverse* (*leading*) i *forward* (*lagging*) *half-strand* polovinama hromozoma [13]

Na Slici 3.11 prikazan je proces formiranja Okazaki fragmenata i njihovo povezivanje u kontinuirani DNK lanac.

Zbog stalnog započinjanja i prekidanja sinteze, *forward half-strand* provodi znatno više vremena u jednonlačanom stanju od *reverse half-strand* polovine hromozo-

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAZENJE POČETNOG REGIONA REPLIKACIJE



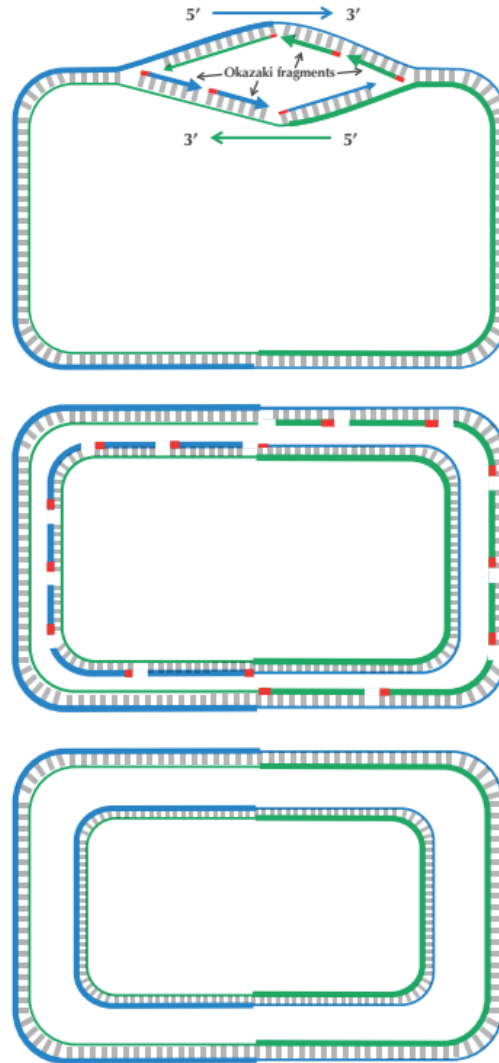
Slika 3.10: Usporedni prikaz asimetrije replikacije DNK na reverse half-strand i forward half-strand polovini hromozoma. Plavi segment označava roditeljski DNK lanac koji služi kao matrica, zeleni segmenti označavaju RNK prajmere, a crveni segmenti novosintetisanu DNK.

ma. Ova razlika između kontinuirane i isprekidane sinteze predstavlja osnovu za nastanak statističke asimetrije između dve polovine hromozoma.

Jednolančana DNK je hemijski nestabilnija i podložnija mutacijama. Posebno je značajan proces deaminacije, pri kojem citozin (C) prelazi u timin (T). Ova mutacija se češće dešava na lancu koji duže ostaje jednolančan. Kao posledica toga, *forward half-strand* vremenom gubi citozin, što dovodi do neravnomerne raspodele nukleotida između dva dela genoma, posebno između baza guanin (G) i citozin (C).

Ova asimetrija u raspodeli nukleotida predstavlja važan signal koji se može iskoristiti za identifikaciju početnog regiona replikacije. Naime, ako posmatramo genom duž njegove dužine, možemo uočiti da u jednom delu dominiraju određene baze, dok se u drugom delu ta raspodela menja. Ta promena u ponašanju direktno je povezana sa pozicijom *oriC*.

Da bi se ovaj fenomen formalizovao, uvodi se funkcija koja prati razliku između broja pojavljivanja baza G i C duž genoma. Ova funkcija predstavlja osnovu *Skew* dijagrama.



Slika 3.11: Nastanak Okazaki fragmenata tokom replikacije i njihovo povezivanje u kontinuirani DNK lanac [14].

3.3.2 Iskrivljeni dijagram (*Skew Diagram*)

GC -*skew* predstavlja meru razlike između broja pojavljivanja baza guanin (G) i citozin (C) u prefiksu DNK sekvence.

Formalno, za poziciju i definišemo:

$$Skew(i) = \#G - \#C,$$

gde $\#G$ i $\#C$ predstavljaju broj pojavljivanja baza G i C u prvih i nukleotida sekvence.

Početna vrednost funkcije je:

$$Skew(0) = 0$$

Konstrukcija *Skew* dijagrama

Skew dijagram se konstruiše prolaskom kroz sekvencu i inkrementalnim ažuriranjem vrednosti funkcije.

Za svaku poziciju važi:

- ako je baza G $\rightarrow$ vrednost se povećava za 1
- ako je baza C $\rightarrow$ vrednost se smanjuje za 1
- ako je baza A ili T $\rightarrow$ vrednost ostaje nepromenjena

Na ovaj način dobija se niz vrednosti koji se može grafički prikazati, gde x -osa predstavlja poziciju u genomu, a y -osa vrednost funkcije $Skew(i)$.

Algoritam 12 prikazuje pseudokod za izračunavanje *GC-skew* vrednosti.

Algoritam 12: Izračunavanje GC-skew

```
Funkcija GC-skew(Genome)
    Skew[0]  $\leftarrow$  0
    for  $i \leftarrow 1$  to |Genome| do
        Skew[ $i$ ]  $\leftarrow$  Skew[ $i - 1$ ]
        if Genome[ $i$ ] = G then
            Skew[ $i$ ]  $\leftarrow$  Skew[ $i$ ] + 1
        end
        else if Genome[ $i$ ] = C then
            Skew[ $i$ ]  $\leftarrow$  Skew[ $i$ ] - 1
        end
    end
    return Skew
```

Interpretacija rezultata

Vrednost funkcije $Skew(i)$ pokazuje karakterističan obrazac prilikom prolaska kroz genom. U jednom delu genoma vrednosti opadaju, dok u drugom počinju da rastu.

Tačka u kojoj funkcija dostiže minimum predstavlja mesto promene ovog trenda i u praksi odgovara početnom regionu replikacije (*oriC*). Intuitivno, minimum

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAZENJE POČETNOG REGIONA REPLIKACIJE

označava prelaz između dva dela genoma sa različitim statističkim osobinama, što je direktna posledica asimetrije procesa replikacije.

Problem minimalnog *skew*-a

Na osnovu prethodne analize može se definisati problem pronalazanja minimalne vrednosti funkcije $Skew(i)$, koji predstavlja osnovu za algoritamsko određivanje lokacije *oriC*. Cilj je pronaći sve pozicije u genomu na kojima funkcija dostiže svoju minimalnu vrednost, jer te pozicije predstavljaju kandidate za početni region replikacije (*oriC*). Problem minimalnog *skew*-a formalno se definiše na sledeći način:

- **Ulaz:** DNK sekvenca *Genome*.
- **Izlaz:** sve pozicije i za koje je vrednost $Skew(i)$ minimalna.

Primer

Na Slici 3.12 prikazan je *GC-skew* dijagram za genom bakterije *Escherichia coli*.

Uočava se karakterističan obrazac u kome vrednost funkcije najpre raste, zatim opada i dostiže minimum, nakon čega ponovo počinje da raste. Ova promena trenda odražava prelaz između dva dela genoma sa različitim statističkim osobinama.

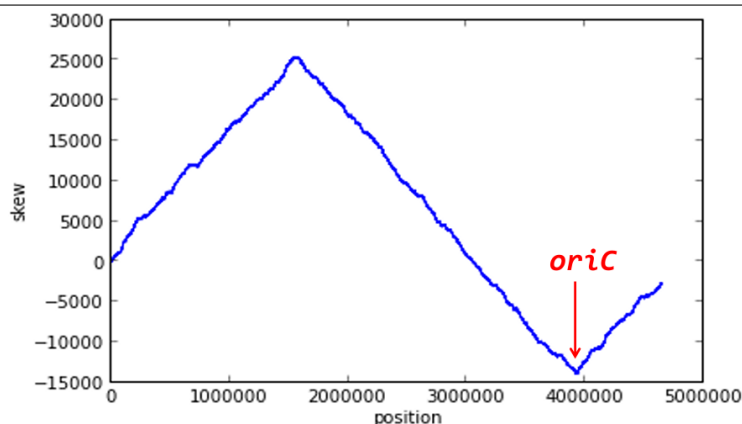
Minimum funkcije dostiže se približno na poziciji od oko 4 miliona nukleotida i predstavlja kandidat za lokaciju početnog regiona replikacije (*oriC*). Razlog za to je što upravo na tom mestu dolazi do promene između *forward* i *reverse half-strand*, odnosno do promene u raspodeli baza G i C.

Takođe, u suprotnom delu genoma funkcija dostiže maksimum, što odgovara regionu približno nasuprot *oriC* na kružnom hromozomu. Ova simetrična struktura dijagrama posledica je same prirode replikacije, koja se odvija u oba smera od početne tačke.

3.3.3 Vremenska složenost

Algoritam za izračunavanje *GC-skew* vrednosti zasniva se na jednom prolazu kroz DNK sekvencu. Za svaku poziciju u genomu vrši se ažuriranje vrednosti funkcije $Skew(i)$ na osnovu prethodne vrednosti, pri čemu se proverava samo trenutni nukleotid.

U svakom koraku izvršava se konstantan broj operacija, što znači da složenost po jednoj iteraciji iznosi $O(1)$.



Slika 3.12: GC-skew dijagram za genom bakterije *Escherichia coli* [12]

Kako se ovaj postupak ponavlja za svih $|Genome|$ pozicija u sekvenci, ukupna vremenska složenost algoritma iznosi:

$$O(|Genome|)$$

Pored toga, potrebno je memorisati niz vrednosti dužine $|Genome|$, pa je prostorna složenost takođe:

$$O(|Genome|).$$

3.3.4 Prednosti i ograničenja

Jedna od glavnih prednosti ovog pristupa jeste njegova jednostavna implementacija, kao i činjenica da se zasniva na linearnom prolazu kroz genom, što omogućava efikasno izračunavanje čak i za veoma duge sekvence. Dodatno, metoda ne zahteva unapred poznate obrasce u DNK, što je čini pogodnom za analizu genoma kod kojih relevantni motivi još uvek nisu identifikovani.

Sa druge strane, GC-skew analiza ima i određena ograničenja. Pre svega, ona se oslanja na statističke osobine koje mogu varirati između različitih organizama, pa ne garantuje uvek precizno određivanje lokacije *oriC* regiona. Takođe, uspešnost metode zavisi od izraženosti asimetrije u raspodeli nukleotida, koja u nekim genomima može biti slabo naglašena. Zbog toga se ovaj pristup, u praksi, najčešće koristi u kombinaciji sa algoritmima zasnovanim na analizi podniski, čime se postiže pouzdanija identifikacija biološki značajnih regiona.

3.4 Približno poklapanje sekvenci i česte reči sa propustima

U prethodnim poglavljima razmatrani su algoritmi zasnovani na tačnom poklapanju podniski u DNK sekvenci. Takav pristup podrazumeva da se funkcionalno značajni obrasci pojavljuju u potpuno identičnom obliku. Međutim, u realnim biološkim sistemima ova pretpostavka često nije zadovoljena zato što su DNK sekvence podložne mutacijama. Tada dolazi do zamene pojedinačnih nukleotida, pa se isti biološki motiv može pojaviti u više varijanti koje se razlikuju u manjem broju pozicija, ali i dalje zadržavaju svoju funkciju.

Zbog toga algoritmi zasnovani isključivo na tačnom poklapanju mogu propustiti značajne obrasce. Da bi se ovaj problem prevazišao, uvodi se model približnog poklapanja sekvenci, koji omogućava poređenje uz dozvoljeni broj odstupanja. Ovaj model predstavlja prirodno proširenje prethodnih pristupa i čini osnovu za rešavanje problema čestih reči sa propustima.

3.4.1 Hamingova distanca

Osnovna mera razlike između sekvenci jednake dužine jeste Hamingova distanca (*Hamming distance*). Za dve sekvence p i q dužine k , Hamingova distanca definiše se kao broj pozicija na kojima se one razlikuju.

Ako su date sekvence

$$p = p_1p_2 \dots p_k \quad \text{i} \quad q = q_1q_2 \dots q_k,$$

onda se Hamingova distanca može formalno zapisati kao:

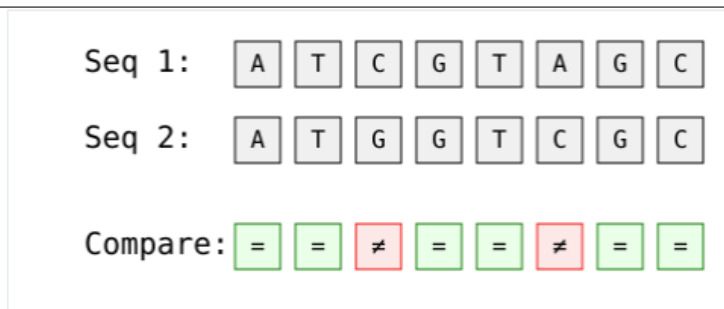
$$\text{HammingDistance}(p, q) = |\{i \mid p_i \neq q_i\}|.$$

Na primeru sa Slike 3.13, sekvence ATCGTAGC i ATGGTCGC razlikuju se u dve pozicije, pa je njihova Hamingova distanca jednaka 2.

Ova mera predstavlja osnovni alat za procenu sličnosti između podniski. Što je Hamingova distanca manja, to su dve sekvence međusobno sličnije.

Algoritam za računanje Hamingove distance

Najjednostavniji način računanja Hamingove distance zasniva se na poređenju odgovarajućih karaktera u dve sekvence iste dužine. Svaki put kada se simboli razlikuju, brojač se uvećava za 1. Algoritam 13 prikazuje pseudokod ove funkcije.



Slika 3.13: Poređenje dve DNK sekvence sa naglašenim razlikama po pozicijama [11]

Algoritam 13: Hamming Distance

Funkcija HammingDistance(p, q)

```

 $d \leftarrow 0$ 
for  $i \leftarrow 1$  to  $|p|$  do
    if  $p_i \neq q_i$  then
         $d \leftarrow d + 1$ 
    end
end
return  $d$ 

```

Ovaj algoritam prolazi kroz obe sekvence samo jednom i pri svakoj iteraciji proverava da li se simboli na istoj poziciji razlikuju. Zbog toga, njegova vremenska složenost je linearna u odnosu na dužinu sekvence.

3.4.2 Približno poklapanje obrazaca (*Approximate Pattern Matching*)

Na osnovu Hamingove distance definiše se problem približnog poklapanja obrazaca. Za razliku od tačnog poklapanja, gde se zahteva potpuno podudaranje sekvenci, ovde se dozvoljava ograničen broj odstupanja.

Problem približnog poklapanja obrazaca formalno se definiše na sledeći način:

- **Ulaz:** niske $Text$ i $Pattern$, i ceo broj d
- **Izlaz:** sve pozicije u $Text$ na kojima se $Pattern$ pojavljuje sa najviše d odstupanja

Drugim rečima, traže se sve podniske dužine $|Pattern|$ čija je Hamingova distanca od obrasca $Pattern$ manja ili jednaka d .

Algoritam za približno poklapanje

Najjednostavniji pristup rešavanju ovog problema zasniva se na kliznom prozoru. Prozor dužine $|Pattern|$ pomera se kroz sekvencu $Text$, a za svaku poziciju računa se Hamingova distanca između obrasca i odgovarajuće podniske. Algoritam 14 prikazuje pseudokod ove funkcije.

Algoritam 14: Approximate Pattern Matching

Funkcija ApproximatePatternMatching($Text, Pattern, d$)

```
positions ← prazan skup
for i ← 0 to |Text| - |Pattern| do
    Pattern' ← Text[i..i + |Pattern| - 1]
    if HammingDistance(Pattern, Pattern') ≤ d then
        dodaj i u positions
    end
end
return positions
```

Objašnjenje rada algoritma

Algoritam sistematski razmatra sve podniske dužine $|Pattern|$ u tekstu. Za svaku od njih vrši se poređenje sa obrascem karakter po karakter, pri čemu se broji broj pozicija na kojima se razlikuju.

Ukoliko broj razlika ne prelazi zadatu vrednost d , smatra se da na toj poziciji postoji približno poklapanje, pa se indeks te pozicije beleži u skup rezultata.

Važno je naglasiti da se i u ovom slučaju uzimaju u obzir preklapajuća pojavljivanja, jer se klizni prozor pomera za jednu poziciju u svakom koraku.

Primer

Posmatrajmo DNK sekvencu:

AACAAGCATAAACATTAAAGAG

i obrazac:

AAAAA,

uz dozvoljeno $d = 2$.

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAZENJE POČETNOG REGIONA REPLIKACIJE

Algoritam koristi klizni prozor dužine 5 i za svaku poziciju i posmatra podnisku $Text[i..i + 4]$, nakon čega računa Hamingovo rastojanje (d_H) u odnosu na obrazac.

Na primer:

- Za $i = 0$: podniska je AACAA. Poređenjem sa AAAAA dobijamo razlike na dve pozicije, pa je $d_H = 2$ i ova pozicija se prihvata.
- Za $i = 1$: podniska je ACAAG. Razlikuje se u tri pozicije, pa je $d_H = 3$ i ova pozicija se odbacuje.
- Za $i = 2$: podniska je CAAGC. Razlikuje se u više od dve pozicije, pa se takođe odbacuje.
- Za $i = 8$: podniska je AAAAC. Razlikuje se u jednoj poziciji, pa je $d_H = 1$ i ova pozicija se prihvata.

Na ovaj način algoritam prolazi kroz celu sekvencu i vraća sve indekse na kojima se pojavljuje približno poklapanje, odnosno gde je Hamingovo rastojanje manje ili jednako d .

3.4.3 Prebrojavanje približnih pojavljivanja

Pored pronalazanja pozicija na kojima se obrazac pojavljuje sa odstupanjima, često je potrebno izračunati i ukupan broj takvih pojavljivanja. Zbog toga se uvodi funkcija:

$$ApproximatePatternCount(Text, Pattern, d),$$

koja vraća broj podniski u tekstu čija je Hamingova distanca od obrasca $Pattern$ manja ili jednaka d .

Algoritam 15: Approximate Pattern Count

Funkcija `ApproximatePatternCount(Text, Pattern, d)`

```
count ← 0
for i ← 0 to |Text| - |Pattern| do
    Pattern' ← Text[i..i + |Pattern| - 1]
    if HammingDistance(Pattern, Pattern') ≤ d then
        count ← count + 1
    end
end
return count
```

Pseudokod je prikazan u Algoritmu 15 i veoma je sličan prethodnom, ali se umesto čuvanja svih pozicija računa samo brojač. Svaki put kada se pronađe podniska koja se razlikuje u najviše d pozicija, brojač se povećava za 1.

Na kraju, algoritam vraća ukupan broj približnih pojavljivanja, odnosno vrednost funkcije *ApproximatePatternCount*.

3.4.4 Vremenska složenost približnog poklapanja

Algoritam za približno poklapanje prolazi kroz sve podniske dužine $|Pattern|$, kojih ima $|Text| - |Pattern| + 1$. Za svaku od njih računa se Hamingova distanca, što zahteva $O(|Pattern|)$ operacija.

Zbog toga ukupna vremenska složenost iznosi:

$$O(|Text| \cdot |Pattern|)$$

Ova složenost je linearna u odnosu na dužinu teksta, ali zavisi i od dužine obrasca. Za kraće obrasce algoritam je efikasan, dok za duže sekvence ili veliki broj poređenja može postati zahtevniji.

3.4.5 Problem čestih reči sa propustima (*Frequent Words with Mismatches*)

Približno poklapanje i približno prebrojavanje pojavljivanja predstavljaju osnovu za definisanje složenijeg problema. Sada cilj nije pronaći poklapanje sa datim obrascem, već pronaći najčešće obrasce kada su dozvoljena odstupanja.

Drugim rečima, ako za dati obrazac možemo da izračunamo koliko puta se pojavljuje u tekstu sa najviše d razlika, tada možemo pokušati da pronađemo one obrasce koji maksimizuju ovu vrednost. Na taj način dolazimo do problema čestih reči sa propustima.

Ovaj problem se može definisati na sledeći način:

- **Ulaz:** niska $Text$, dužina k i maksimalan broj dozvoljenih odstupanja d
- **Izlaz:** svi k -grami koji maksimizuju vrednost funkcije

$$ApproximatePatternCount(Text, Pattern, d)$$

Za razliku od standardnog problema čestih reči, ovde frekvencija obrasca ne zavisi od broja tačnih poklapanja, već od broja približnih poklapanja. To znači da

obrazac sa propustima koji je najčešći ne mora nužno da se pojavljuje u tekstu u svom tačnom obliku. Upravo to čini ovaj problem složenijim i istovremeno biološki značajnijim.

3.4.6 d -susedstvo obrasca

Kod problema tačnih poklapanja bilo je dovoljno posmatrati one k -grame koji se zaista pojavljuju u tekstu. Ovde to više nije slučaj. Razlog je to što obrazac sa propustima koji na kraju bude najčešći ne mora biti eksplicitno prisutan u tekstu. Moguće je da se više podniski iz teksta razlikuje od nekog zajedničkog motiva u jednoj ili dve pozicije, i da upravo taj „centralni“ motiv ima najveću približnu frekvenciju, iako se sam nikada ne pojavljuje tačno. Zbog toga je potrebno razmotriti ne samo postojeće podniske, već i njihove varijacije.

Za dati obrazac *Pattern*, njegovo d -susedstvo, označeno sa $Neighbors(Pattern, d)$, predstavlja skup svih sekvenci iste dužine koje se razlikuju od *Pattern* u najviše d pozicija.

Na primer, ako je dozvoljeno jedno odstupanje, tada susedstvo nekog obrasca sadrži sve sekvence koje se od njega razlikuju u najviše jednoj poziciji. Kako broj dozvoljenih odstupanja raste, susedstvo postaje sve veće.

Pojam susedstva je ključan zato što omogućava da se pretraga ograniči na skup kandidata koji su biološki smisleni, odnosno na sekvence koje su dovoljno bliske onima koje se zaista pojavljuju u tekstu.

Generisanje d -susedstva

Funkcija $Neighbors(Pattern, d)$ može se implementirati rekursivno. Osnovna ideja rekursije zasniva se na tome da se prvo generišu susedi sufiksa obrasca, a zatim se u zavisnosti od njihove Hamingove distance dodaje odgovarajući simbol na početak. Na ovaj način se korak po korak generišu sve sekvence koje zadovoljavaju uslov da se razlikuju od početnog obrasca u najviše d pozicija. Pseudokod prikazan je u Algoritmu 16.

Objašnjenje rada funkcije *Neighbors*

Ako je broj dozvoljenih odstupanja jednak nuli, jedini sused obrasca jeste sam obrazac. Ako je dužina obrasca jednaka 1, tada su svi simboli iz alfabeta mogući susedi.

Algoritam 16: Neighbors

```

Funkcija Neighbors(Pattern, d)
    if d = 0 then
        | return {Pattern}
    end
    if |Pattern| = 1 then
        | return {A, C, G, T}
    end
    Neighborhood  $\leftarrow$  prazan skup
    SuffixNeighbors  $\leftarrow$  Neighbors(Suffix(Pattern), d)
    foreach Text  $\in$  SuffixNeighbors do
        | if HammingDistance(Suffix(Pattern), Text) < d then
            | foreach x  $\in$  {A, C, G, T} do
                | | dodaj x · Text u Neighborhood
            | end
        | end
        | else
            | | dodaj FirstSymbol(Pattern) · Text u Neighborhood
        | end
    end
    return Neighborhood

```

U opštem slučaju najpre se rekurzivno generišu susedi sufiksa obrasca. Zatim se za svaki takav sused proverava da li je njegova Hamingova distanca od originalnog sufiksa strogo manja od d . Ako jeste, na početak se može dodati bilo koji simbol, jer još uvek postoji prostor za dodatnu razliku. U suprotnom, mora se zadržati originalni prvi simbol, kako ukupan broj odstupanja ne bi prešao dozvoljenu vrednost.

3.4.7 Algoritam za česte reči sa propustima

Jedan od efikasnijih pristupa za rešavanje problema čestih reči sa propustima zasniva se na tome da se ne razmatra svih 4^k mogućih obrazaca, već samo oni koji pripadaju susedstvu nekog k -grama iz teksta.

U tu svrhu koriste se nizovi *Close* i *FrequencyArray*. Niz *Close* označava da li je dati obrazac kandidat, dok *FrequencyArray* čuva broj njegovih približnih pojavljivanja. Algoritam 17 prikazuje pseudokod ove funkcije *Frequent Words with Mismatches*.

Algoritam 17: Frequent Words with Mismatches

Funkcija *FrequentWordsWithMismatches*(*Text*, *k*, *d*)

```

    FrequentPatterns  $\leftarrow$  prazan skup
    for  $i \leftarrow 0$  to  $4^k - 1$  do
         $Close[i] \leftarrow 0$ 
         $FrequencyArray[i] \leftarrow 0$ 
    end
    for  $i \leftarrow 0$  to  $|Text| - k$  do
         $Neighborhood \leftarrow Neighbors(Text[i..i + k - 1], d)$ 
        foreach  $Pattern \in Neighborhood$  do
             $index \leftarrow PatternToNumber(Pattern)$ 
             $Close[index] \leftarrow 1$ 
        end
    end
    for  $i \leftarrow 0$  to  $4^k - 1$  do
        if  $Close[i] = 1$  then
             $Pattern \leftarrow NumberToPattern(i, k)$ 
             $FrequencyArray[i] \leftarrow$ 
                 $ApproximatePatternCount(Text, Pattern, d)$ 
        end
    end
     $maxCount \leftarrow$  maksimalna vrednost u  $FrequencyArray$ 
    for  $i \leftarrow 0$  to  $4^k - 1$  do
        if  $FrequencyArray[i] = maxCount$  then
             $Pattern \leftarrow NumberToPattern(i, k)$ 
            dodaj  $Pattern$  u  $FrequentPatterns$ 
        end
    end
    return  $FrequentPatterns$ 

```

Objašnjenje rada algoritma

U prvom delu algoritma inicijalizuju se pomoćni nizovi. Nakon toga prolazi se kroz sve k -grame u tekstu, i za svaki od njih generiše se odgovarajuće d -susedstvo. Svaki obrazac iz tog susedstva označava se kao potencijalni kandidat.

U drugom delu algoritma prolazi se kroz sve moguće indekse. Ako je neki obrazac označen kao kandidat, za njega se računa vrednost funkcije *ApproximatePatternCount*, odnosno broj približnih pojavljivanja u tekstu.

Na kraju se pronalazi maksimalna vrednost u nizu frekvencija i izdvajaju se svi obrasci koji postižu tu vrednost.

Važno je uočiti da se funkcija *ApproximatePatternCount* ne poziva za svih 4^k mogućih obrazaca, već samo za one koji pripadaju susedstvu nekog obrasca iz teksta. Time se postiže značajna ušteda u odnosu na potpuno iscrpnu pretragu.

3.4.8 Uzimanje u obzir obrnutih komplementa

U analizi DNK sekvenci nije dovoljno posmatrati samo jednu orijentaciju obrasca. Sekvenca i njen obrnuti komplement mogu imati istu biološku funkciju zato što se proteini mogu vezivati za motive prisutne na oba komplementarna lanca.

Zbog toga, ovaj problem se može dodatno proširiti tako da se za svaki obrazac posmatra i njegov obrnuti komplement. U tom slučaju frekvencija obrasca računa se kao zbir:

$$\begin{aligned} & \text{ApproximatePatternCount}(\text{Text}, \text{Pattern}, d) \\ & + \\ & \text{ApproximatePatternCount}(\text{Text}, \text{ReverseComplement}(\text{Pattern}), d) \end{aligned}$$

Na taj način se objedinuju obe moguće orijentacije iste informacije zapisane u DNK.

Problem čestih reči sa propustima i obrnutim komplementima

Kada se u obzir uzmu i obrnuti komplementi, problem se može definisati na sledeći način:

- **Ulaz:** DNK sekvenca *Text*, dužina k i maksimalan broj odstupanja d
- **Izlaz:** svi k -grami koji maksimizuju zbir približnih frekvencija svog oblika i svog obrnutog komplementa

Ova formulacija predstavlja realističniji model biološkog problema, jer uzima u obzir i prirodnu varijabilnost sekvenci i njihovu komplementarnost.

3.4.9 Vremenska složenost

Najveći izazov ovog pristupa jeste dimenzija d -susedstva. Za dati obrazac dužine k , broj suseda raste sa povećanjem broja dozvoljenih odstupanja d . Taj broj se može precizno izraziti formulom:

$$\sum_{i=0}^d \binom{k}{i} \cdot 3^i,$$

jer se za svaku od i izmenjenih pozicija bira jedna od tri moguće alternativne baze.

Zbog toga, generisanje d -susedstva za sve k -grame u tekstu, kojih ima približno $|Text| - k + 1$, zahteva značajan broj operacija. Ukupan broj generisanih kandidata reda je veličine:

$$O(|Text| \cdot 3^d \cdot k^d)$$

Dodatni trošak nastaje u drugoj fazi algoritma, gde se za svakog različitog kandidata računa vrednost funkcije *ApproximatePatternCount*. Neka je broj različitih kandidata označen sa C . Kako funkcija *ApproximatePatternCount* ima vremensku složenost $O(|Text| \cdot k)$, ukupni trošak ove faze iznosi:

$$O(C \cdot |Text| \cdot k),$$

gde važi:

$$C \leq \min(4^k, |Text| \cdot 3^d \cdot k^d).$$

Pored toga, algoritam prolazi kroz nizove *Close* i *FrequencyArray* dužine 4^k , pri čemu rekonstrukcija odgovarajućih obrazaca pomoću funkcije *NumberToPattern* zahteva ukupno:

$$O(4^k \cdot k).$$

Kombinovanjem svih koraka dobija se ukupna vremenska složenost:

$$O(|Text| \cdot 3^d \cdot k^d + C \cdot |Text| \cdot k + 4^k \cdot k).$$

Ovaj rezultat pokazuje da algoritam ima visoku vremensku složenost, koja brzo raste sa povećanjem parametara k i d . Iako pristup zasnovan na susedstvu značajno smanjuje broj kandidata u odnosu na potpuno iscrpnu pretragu, problem i dalje ostaje računarski zahtevan.

3.4.10 Optimizovana implementacija za česte reči sa propustima

Prethodno opisani pristup za rešavanje problema čestih reči sa propustima zasniva se na korišćenju nizova dimenzije 4^k , što može predstavljati značajan nedostatak u pogledu memorijske efikasnosti. Kako vrednost k raste, broj svih mogućih k -grama eksponencijalno se povećava, pa čuvanje informacija za svaki od njih postaje nepraktično.

Da bi se ovaj problem ublažio, može se primeniti optimizovani pristup zasnovan na korišćenju heš mape. Umesto da se unapred razmatraju svi mogući obrasci, u memoriji se čuvaju samo oni koji se pojavljuju u d -susedstvu nekog k -grama iz posmatranog teksta.

Ideja je da se prolazi kroz tekst i za svaki izdvojeni k -gram generiše njegovo d -susedstvo. Za svaki obrazac iz tog susedstva proverava se da li je već obrađen. Ako nije, izračunava se broj njegovih približnih pojavljivanja u tekstu i čuva u rečniku. Ključ u rečniku predstavlja obrazac, dok vrednost predstavlja broj njegovih približnih pojavljivanja.

Na ovaj način svaki kandidat se obrađuje najviše jednom, čime se izbegava nepotrebno ponavljanje izračunavanja, kao i čuvanje frekvencija za obrasce koji se uopšte ne pojavljuju kao relevantni kandidati.

Algoritam

Algoritam prolazi kroz tekst, generiše d -susedstvo za svaki k -gram i za svakog suseda računa približnu frekvenciju samo ako on već nije evidentiran u rečniku. Na kraju se iz rečnika izdvajaju svi obrasci koji imaju maksimalnu frekvenciju i oni predstavljaju najzastupljenije k -grame sa dozvoljenim odstupanjima [15]. Pseudokod je prikazan u Algoritmu 18.

Prednosti pristupa

Glavna prednost ovog pristupa je smanjenje memorijske potrošnje, jer se izbegava korišćenje nizova dimenzije 4^k . Umesto toga, čuvaju se samo oni obrasci koji su zaista relevantni kandidati.

Pored toga, izbegava se i prolazak kroz sve moguće obrasce, pa se fokus stavlja isključivo na one koji potiču iz stvarnih delova teksta. Iako dominantni deo vremenske složenosti ostaje vezan za generisanje d -susedstva i računanje funkcije

Algoritam 18: Frequent Words with Mismatches (Dictionary)

Funkcija `FrequentWordsWithMismatchesDictionary(Text, k, d)`

```

    FrequentPatterns  $\leftarrow$  prazan skup
    FrequencyMap  $\leftarrow$  prazna heš mapa
    for  $i \leftarrow 0$  to  $|Text| - k$  do
        Pattern  $\leftarrow Text[i..i + k - 1]$ 
        Neighborhood  $\leftarrow Neighbors(Pattern, d)$ 
        foreach Neighbor  $\in Neighborhood$  do
            if Neighbor nije ključ u FrequencyMap then
                FrequencyMap[Neighbor]  $\leftarrow$ 
                    ApproximatePatternCount(Text, Neighbor, d)
            end
        end
    end
    maxCount  $\leftarrow$  maksimalna vrednost u FrequencyMap
    foreach Pattern u FrequencyMap do
        if FrequencyMap[Pattern] = maxCount then
            dodaj Pattern u FrequentPatterns
        end
    end
    return FrequentPatterns

```

ApproximatePatternCount za različite kandidate, ovaj pristup je u praksi memorijski efikasniji i jednostavniji za implementaciju.

3.5 Ograničenja i otvoreni problemi

U prethodnim poglavljima predstavljani su algoritmi za analizu DNK sekvenci zasnovani na frekvenciji pojavljivanja podniski, njihovom lokalnom grupisanju i dozvoljenim odstupanjima. Iako ovi pristupi pružaju značajne uvide i omogućavaju identifikaciju potencijalno važnih obrazaca, njihova primena u realnim biološkim podacima otkriva niz ograničenja.

U nastavku ćemo razmotriti dodatne aspekte i otvorene probleme koji proizlaze iz ovih metoda.

3.5.1 Ograničenja frekvencijskih pristupa

Algoritmi poput *Frequent Words* i *Frequent Words with Mismatches* zasnivaju se na pretpostavci da su biološki značajni motivi ujedno i najčešće zastupljeni u

sekvenci. Međutim, u praksi ova pretpostavka nije uvek validna.

U dugim DNK sekvencama određeni k -grami mogu se pojavljivati često isključivo kao posledica slučajnosti. Sa druge strane, funkcionalno važni motivi mogu biti prisutni u manjem broju kopija ili sa varijacijama, zbog čega ih frekvencijski pristupi ne detektuju pouzdano.

Uvođenjem dozvoljenih odstupanja povećava se fleksibilnost algoritma, ali istovremeno dolazi do značajnog povećanja broja kandidata, što može dovesti do pojave velikog broja lažno pozitivnih rezultata.

Kombinatorna složenost i praktični izazovi

Jedan od glavnih izazova jeste dimenzija prostora pretrage. Broj mogućih k -grama iznosi 4^k , a uz dozvoljena odstupanja dodatno raste zbog generisanja d -susedstva.

Ova kombinatorna eksplozija čini problem računarski zahtevnim, posebno za veće vrednosti parametara k i d . Iako optimizacije, poput korišćenja niza frekvencija i ograničavanja na relevantne kandidate, smanjuju broj operacija, algoritmi i dalje mogu biti spori za velike genomske sekvence.

Statistički aspekti i verovatnoća obrazaca

Važno pitanje koje se prirodno nameće jeste kako razlikovati obrasce koji su biološki značajni od onih koji su nastali slučajno. U tu svrhu neophodno je posmatrati verovatnoću pojavljivanja određenih podniski u sekvenci.

Ukoliko se neki obrazac pojavljuje češće nego što bi se očekivalo na osnovu slučajnog modela, može se smatrati potencijalno značajnim. Međutim, precizno modeliranje ovih verovatnoća predstavlja dodatni izazov i zahteva statističke metode koje prevazilaze osnovne algoritamske pristupe.

3.5.2 Struktura DNK i dodatni biološki faktori

Analiza DNK sekvenci ne zavisi samo od frekvencije podniski, već i od njihove biološke interpretacije. Na primer, značajnu ulogu imaju:

- komplementarnost DNK lanaca i postojanje obrnutih komplementa,
- pravac ($5' \rightarrow 3'$) u kome se sekvence čitaju,

- preklapanje podniski, koje može uticati na statističku raspodelu obrazaca.

Ovi faktori dodatno komplikuju analizu i ukazuju na to da algoritamski rezultati moraju biti interpretirani u biološkom kontekstu.

Otvoreni problemi u pronalaženju motiva

Jedan od najvažnijih otvorenih problema jeste pronalaženje skrivenih motiva u DNK sekvencama. Za razliku od prethodnih problema, gde se analizirala jedna sekvenca, ovde se često posmatra više sekvenci i traži zajednički obrazac koji se u njima pojavljuje, ali ne nužno u identičnom obliku.

Ovaj problem je složen zato što:

- motivi mogu sadržati mutacije,
- ne pojavljuju se na istim pozicijama,
- mogu biti maskirani šumom u podacima,
- nisu nužno najčešći obrasci u pojedinačnoj sekvenci.

Zbog toga standardni pristupi zasnovani na frekvenciji nisu dovoljni, već je potrebno razvijati naprednije algoritme koji kombinuju različite kriterijume.

Primeri otvorenih problema

U okviru savremene bioinformatike izdvajaju se i specifični problemi vezani za različite organizme:

- identifikacija više početnih regiona replikacije u nekim bakterijama,
- pronalaženje *oriC* regiona kod arheja, gde struktura genoma odstupa od bakterijske,
- analiza složenih eukariotskih genoma, kao što je kvasac, gde su regulatorni mehanizmi znatno kompleksniji.

Ovi problemi ukazuju na to da univerzalno rešenje ne postoji i da metode moraju biti prilagođene konkretnom biološkom sistemu.

Iako predstavljeni algoritmi čine osnovu za analizu DNK sekvenci, njihova ograničenja ukazuju na potrebu za daljim razvojem metoda u bioinformatici. Kombinacija

GLAVA 3. ALGORITAMSKI PRISTUPI ZA PRONALAŽENJE POČETNOG REGIONA REPLIKACIJE

algoritamskih tehnika, statističkih modela i biološkog znanja predstavlja ključ za uspešno rešavanje ovih problema. Zbog toga se savremena istraživanja fokusiraju na razvoj složenijih pristupa koji mogu pouzdanije identifikovati funkcionalno značajne motive u genomu.

Glava 4

Elektronska lekcija

U ovom poglavlju opisana je elektronska lekcija razvijena kao praktični deo rada. Lekcija je namenjena interaktivnom prikazu algoritama koji se koriste pri analizi DNK sekvenci i pronalaženju potencijalnog početnog regiona replikacije. Pored teorijskog objašnjenja pojmova, aplikacija omogućava korisniku da unese DNK sekvencu, pokrene odgovarajuće algoritme i prati njihove rezultate kroz vizuelni prikaz.

Elektronska lekcija se sastoji iz dve komponente: klijentske i serverske. Klijentska komponenta predstavlja korisnički interfejs aplikacije i implementirana je pomoću *Next.js* [3] okvira, uz korišćenje biblioteka *React* [6] i *Node.js* [4]. Serverska komponenta implementirana je u programskom jeziku *Python* [5], uz korišćenje *FastAPI*[1] okvira. Izvorni kod aplikacije nalazi se u javnom *GitHub* [2] repozitorijumu na adresi:

<https://github.com/dunjaci/master-rad>

Aplikacija je javno dostupna preko sledeće adrese:

<https://master-rad-seven.vercel.app>

Serverska komponenta dostupna je preko adrese:

<https://master-rad-rynl.onrender.com/docs>

4.1 Arhitektura aplikacije

Aplikacija je organizovana kao sistem sa odvojenom klijentskom i serverskom komponentom. Klijentska komponenta je zadužena za prikaz stranica, unos podata-

ka, navigaciju kroz lekciju i vizuelizaciju rezultata. Serverska komponenta obrađuje zahteve koje šalje klijentska aplikacija i izvršava algoritamsku logiku.

Ovakva organizacija omogućava jasnu podelu odgovornosti. Korisnički interfejs se razvija nezavisno od serverskog dela, dok se algoritmi i obrada podataka centralizuju u serverskoj komponenti. Na taj način se izbegava dupliranje algoritamske logike u klijentskoj aplikaciji, a rezultati koji se prikazuju korisniku dobijaju se preko definisanih API ruta.

Komunikacija između klijentske i serverske komponente ostvaruje se slanjem HTTP zahteva. Na primer, kada korisnik unese DNK sekvencu i izabere određene parametre, klijentska aplikacija šalje zahtev serverskoj komponenti, koja izvršava odgovarajući algoritam i vraća rezultat u JSON formatu.

4.2 Korišćene tehnologije

Klijentski deo aplikacije implementiran je korišćenjem **Next.js** okvira. *Next.js* omogućava izradu *React* aplikacija sa organizovanom strukturom stranica, jednostavnim rutiranjem i podrškom za produkcionu *build*. Za stilizovanje korisničkog interfejsa korišćen je **Tailwind CSS** [8], što omogućava jednostavno kreiranje responsivnih i preglednih veb stranica.

Serverski deo aplikacije implementiran je pomoću **FastAPI** okvira. *FastAPI* je pogodan za izradu *REST API*-ja jer omogućava jednostavno definisanje ruta, validaciju ulaznih podataka i automatski generisanu dokumentaciju API-ja. U ovoj aplikaciji serverska komponenta sadrži implementacije algoritama i vraća rezultate klijentskoj komponenti.

Za kontrolu verzija i čuvanje izvornog koda korišćen je **Git**, dok je repozitorijum postavljen na **GitHub**. Za javno postavljanje aplikacije korišćene su platforme **Render** [7] i **Vercel** [9]. Serverska komponenta postavljena je na *Render*, dok je klijentska komponenta postavljena na *Vercel*.

4.3 Pokretanje aplikacije u lokalnom okruženju

Aplikacija se može pokrenuti lokalno pokretanjem serverske i klijentske komponente u odvojenim terminalima. Serverska komponenta se nalazi u direktorijumu *backend*, dok se klijentska komponenta nalazi u direktorijumu *frontend*.

4.3.1 Pokretanje serverske komponente

Za pokretanje serverske komponente potrebno je pozicionirati se u direktorijum *backend*. Nakon toga se instaliraju potrebne biblioteke iz datoteke *requirements.txt*. Serverska aplikacija se pokreće pomoću alata *uvicorn*.

```
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Nakon pokretanja, serverska komponenta dostupna je na adresi:

`http://localhost:8000`

Ispravnost rada serverske komponente može se proveriti otvaranjem sledeće adrese:

`http://localhost:8000/api/health`

Ukoliko je server uspešno pokrenut, dobija se odgovor:

```
{"status": "ok"}
```

4.3.2 Pokretanje klijentske komponente

Za pokretanje klijentske komponente potrebno je pozicionirati se u direktorijum *frontend*. Zatim se instaliraju potrebni paketi i pokreće razvojni server.

```
cd frontend
npm install
npm run dev
```

Nakon pokretanja, klijentskoj aplikaciji se može pristupiti preko adrese:

`http://localhost:3000`

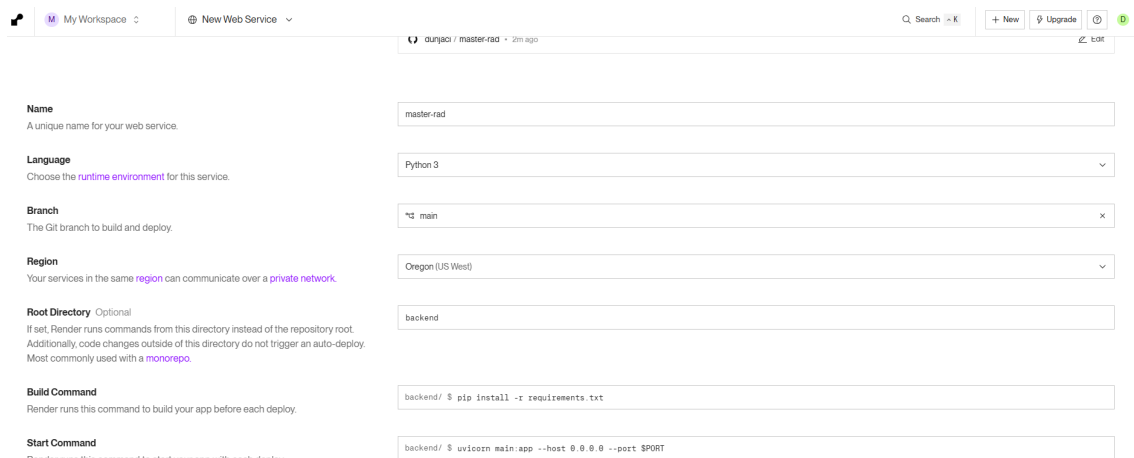
U lokalnom okruženju klijentska aplikacija podrazumevano šalje zahteve na adresu `http://localhost:8000`. Na taj način se lokalno pokrenuti frontend povezuje sa lokalno pokrenutim backend serverom.

4.4 Postavljanje aplikacije na javno dostupne servise

Da bi elektronska lekcija bila dostupna korisnicima preko javnog linka, aplikacija je postavljena na dve platforme. Serverska komponenta postavljena je na *Render*, dok je klijentska komponenta postavljena na *Vercel*. Ovakav pristup odgovara arhitekturi projekta jer su klijentski i serverski deo odvojeni.

4.4.1 Postavljanje serverske komponente na *Render*

Serverska komponenta postavljena je na platformu *Render* kao *Web Service*. *Render* je povezan sa *GitHub* repozitorijumom, čime je omogućeno da se nova verzija serverske komponente automatski postavi nakon izmena u repozitorijumu.

The image shows the 'New Web Service' configuration page on the Render platform. The form is titled 'New Web Service' and is for a repository named 'dunpacs / master-rad'. It contains several fields for configuring the service: 'Name' (master-rad), 'Language' (Python 3), 'Branch' (*2 main), 'Region' (Oregon (US West)), 'Root Directory' (Optional, backend), 'Build Command' (backend/ \$ pip install -r requirements.txt), and 'Start Command' (backend/ \$ uvicorn main:app --host 0.0.0.0 --port \$PORT). Each field has a description and a text input area.

Slika 4.1: Podešavanja za deploy serverske komponente na *Render* platformu

Kao što je prikazano na Slici 4.1, prilikom kreiranja servisa podešeni su sledeći parametri:

Root Directory: backend

Runtime: Python

Build Command: pip install -r requirements.txt

Start Command: uvicorn main:app --host 0.0.0.0 --port \$PORT

Parametar Root Directory označava da se serverska komponenta nalazi u direktorijumu backend. Komanda za izgradnju instalira sve zavisnosti navedene u

datoteci `requirements.txt`, dok komanda za pokretanje startuje *FastAPI* aplikaciju.

Nakon uspešnog postavljanja, *Render* generiše javni URL serverske komponente:

`https://master-rad-rynl.onrender.com`

4.4.2 Postavljanje klijentske komponente na *Vercel*

Klijentska komponenta postavljena je na platformu *Vercel*. *Vercel* je posebno pogodan za *Next.js* aplikacije jer automatski prepoznaje strukturu projekta i izvršava proces izgradnje aplikacije.

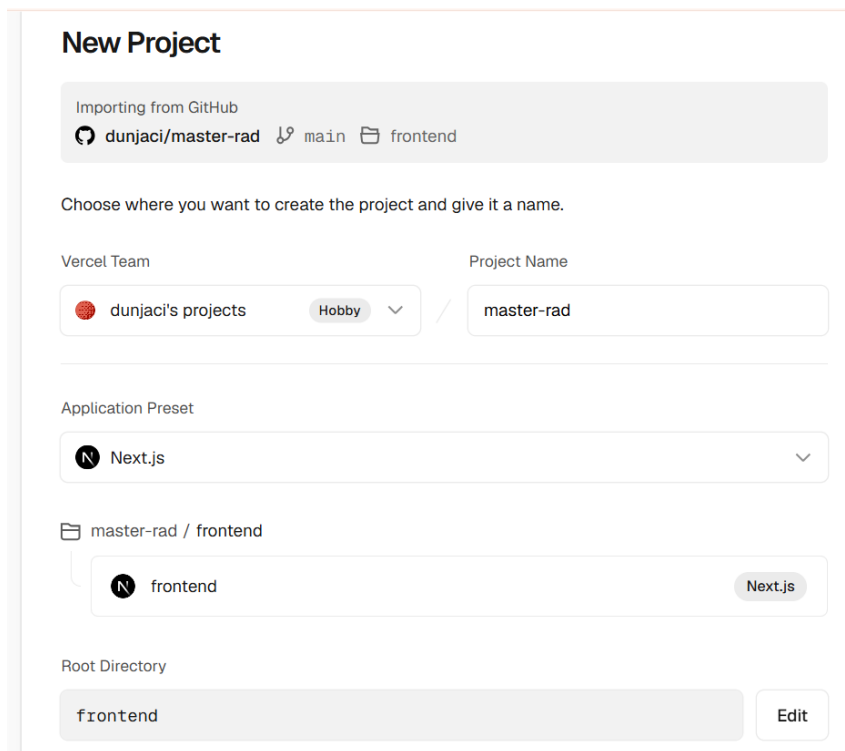
Slike 4.2 i 4.3 prikazuju podešavanja parametara prilikom kreiranja *Vercel* projekta:

Root Directory: `frontend`

Framework Preset: `Next.js`

Build Command: `npm run build`

Install Command: `npm install`



New Project

Importing from GitHub
dunjaci/master-rad main frontend

Choose where you want to create the project and give it a name.

Vercel Team: dunjaci's projects Hobby / Project Name: master-rad

Application Preset: Next.js

master-rad / frontend
frontend Next.js

Root Directory: frontend Edit

Slika 4.2: Podešavanja za deploy klijentske komponente na Vercel platformu

Pošto se klijentska aplikacija nalazi u direktorijumu **frontend**, taj direktorijum je označen kao korenski direktorijum projekta. Nakon uspešnog postavljanja, *Vercel* generiše javnu adresu aplikacije:

`https://master-rad-seven.vercel.app`

4.4.3 Povezivanje klijentske i serverske komponente

Povezivanje klijentske i serverske komponente ostvareno je pomoću promenljivih okruženja. Klijentska aplikacija koristi promenljivu `NEXT_PUBLIC_API_BASE_URL`, koja sadrži adresu serverske komponente:

`NEXT_PUBLIC_API_BASE_URL=https://master-rad-rynl.onrender.com`

Na osnovu ove promenljive, klijentska aplikacija zna na koju adresu treba da šalje API zahteve. U lokalnom okruženju, ukoliko promenljiva nije definisana, koristi se podrazumevana adresa `http://localhost:8000`.

Build Command ⓘ

npm run build

Output Directory ⓘ

Next.js default

Install Command ⓘ

npm install

Environment Variables

Key

NEXT_PUBLIC_API_BASE_URL

Value

https://master-rad-rynl.onrender.com

Environments

Production and Preview

Import .env or paste the .env contents [Learn more](#)

+ Add More

Deploy

Slika 4.3: Podešavanja za deploy klijentske komponente na Vercel platformu

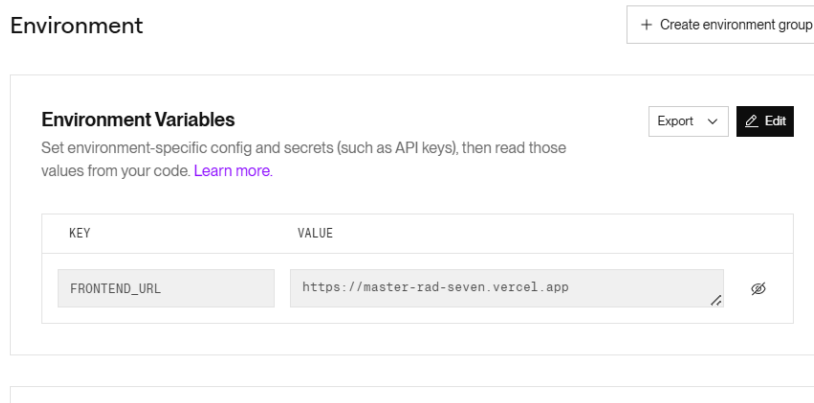
Sa druge strane, kao što je prikazano na Slici 4.4, serverska komponenta koristi promenljivu `FRONTEND_URL`, kojom se definiše sa kog domena je dozvoljeno slanje zahteva ka backend serveru:

```
FRONTEND_URL=https://master-rad-seven.vercel.app
```

Ova promenljiva koristi se za podešavanje *CORS* politike. Time se omogućava da produkciona verzija klijentske aplikacije komunicira sa serverskom komponentom, dok lokalna verzija i dalje može da koristi `http://localhost:3000`.

Nakon povezivanja komponenti, tok komunikacije može se predstaviti na sledeći način:

```
Vercel frontend -> Render backend API -> rezultat se vraća korisniku
```



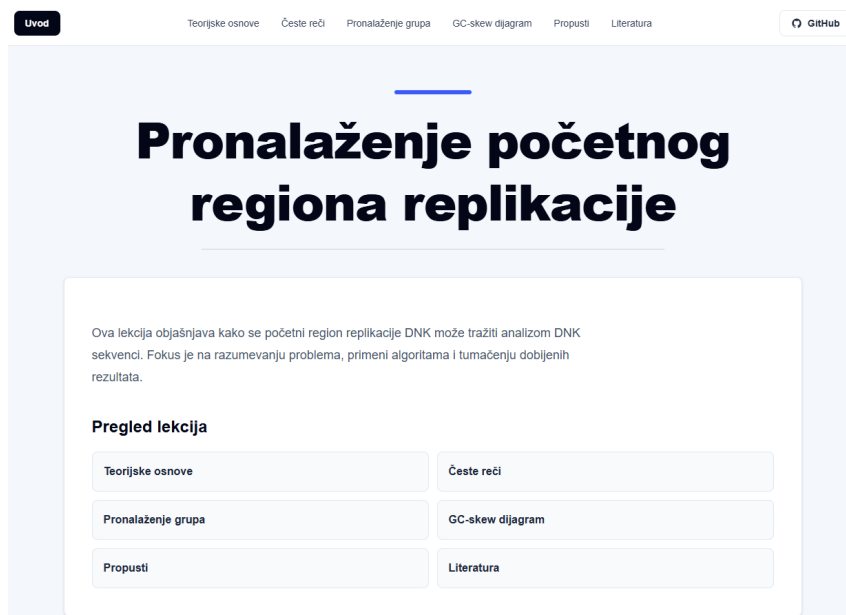
Slika 4.4: Podešavanja za povezivanje klijentske i serverske komponente

4.5 Funkcionalnosti elektronske lekcije

Elektronska lekcija se sastoji iz više stranica koje zajedno prikazuju teorijski uvod, algoritme i literaturu. Na vrhu aplikacije nalazi se navigacioni meni preko kog korisnik može da pristupi svim delovima lekcije. Navigacija sadrži stranice: Uvod, Teorijske osnove, Česte reči, Pronalaženje grupa, *GC-skew* dijagram, Propusti i Literatura. U navigaciji se nalazi i link ka GitHub repozitorijumu.

4.5.1 Početna stranica

Početna stranica predstavlja ulaznu tačku aplikacije (Slika 4.5). Na njoj je prikazan naslov lekcije i kratak pregled dostupnih celina. Korisnik sa ove stranice može da pređe na teorijske osnove, prikaze algoritama ili literaturu.



Slika 4.5: Početna stranica elektronske lekcije

4.5.2 Teorijske osnove

Stranica Teorijske osnove, prikazana na Slici 4.6, uvodi biološke pojmove potrebne za razumevanje algoritama. Objašnjeni su struktura DNK molekula, proces replikacije, početni region replikacije, DnaA boksovi i pojam obrnutog komplementa. Klikom na sekciju prikazuju se interaktivne kartice sa detaljnim objašnjenjima pojmova. Ovaj deo lekcije povezuje biološku motivaciju sa algoritamskim problemima koji se kasnije obrađuju.

Uvod
Teorijske osnove
Česte reči
Pronalaženje grupa
GC-skew dijagram
Propusti
Literatura
GitHub

TEORIJSKE OSNOVE

Biološka osnova problema

Dezoksiribonukleinska kiselina (DNK) je osnovni nosilac genetičke informacije kod živih organizama. Jedan od značajnih izazova u bioinformatici predstavlja identifikacija funkcionalno relevantnih regiona unutar DNK sekvenci, pri čemu se ova analiza često zasniva na primeni algoritamskih metoda.

Posebno interesantan problem predstavlja određivanje početnog regiona replikacije (oriC), koji ima ključnu ulogu u procesu umnožavanja genetičkog materijala. U okviru velikih genoma, koji mogu sadržati milione nukleotida, ovaj region je relativno kratak i nije eksplicitno označen. Stoga se njegovo identifikovanje zasniva na analizi obrazaca u DNK sekvenci i prepoznavanju karakterističnih signala koji ukazuju na početak replikacije.

Jedan od ključnih uvida jeste da se u ovim regionima često pojavljuju ponavljajuće sekvence, kao i specifične statističke nepravilnosti u raspodeli nukleotida. Ove osobine omogućavaju primenu algoritamskih metoda za identifikaciju kandidata za oriC region.

Struktura DNK

DNK je molekul koji se sastoji od dva međusobno povezana lanca koji formiraju

Replikacija DNK

Replikacija DNK predstavlja proces umnožavanja genetskog materijala koji se

Početni region replikacije (oriC)

Početni region replikacije, označen kao oriC, predstavlja specifičan segment DNK

Slika 4.6: Stranica sa teorijskim osnovama

4.5.3 Česte reči

Stranica Česte reči prikazuje algoritamski problem pronalaženja najčešćih k-grama u zadatoj DNK sekvenci. Korisnik može da unese sekvencu i vrednost parametra k . Aplikacija zatim prikazuje rezultat i omogućava praćenje rada različitih pristupa.

Na ovoj stranici predstavljeni su naivni algoritam, optimizacija nizom frekvencija, optimizacija heš mapom i pristup sortiranjem. Za svaki pristup prikazana je vremenska složenost, objašnjenje i programski kod u jeziku *Python*. Vizuelni prikaz omogućava korisniku da prati trenutni k-gram, njegovu poziciju i način na koji algoritam obrađuje ulaznu sekvencu (Slika 4.7).

VIZUELNI PRIKAZ

Pokretanje algoritma

Naivni algoritam

DNK sekvenca

ACTGACTCCACCCC

k

3 Dužina teksta: 15

A C T G A C T C C A C C C

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

Korak: 3/13

Trenutni k-mer: TGA

Broj pojavljivanja: 1

Prethodni korak Pokreni od početka Sledeći korak

Šta radi naivni algoritam?

Algoritam posmatra podnisku TGA na poziciji 2. Zatim ponovo prolazi kroz ceo tekst i broji sva njena pojavljivanja. Pojavljuje se na pozicijama 2.

Pojavljivanja trenutnog k-mera

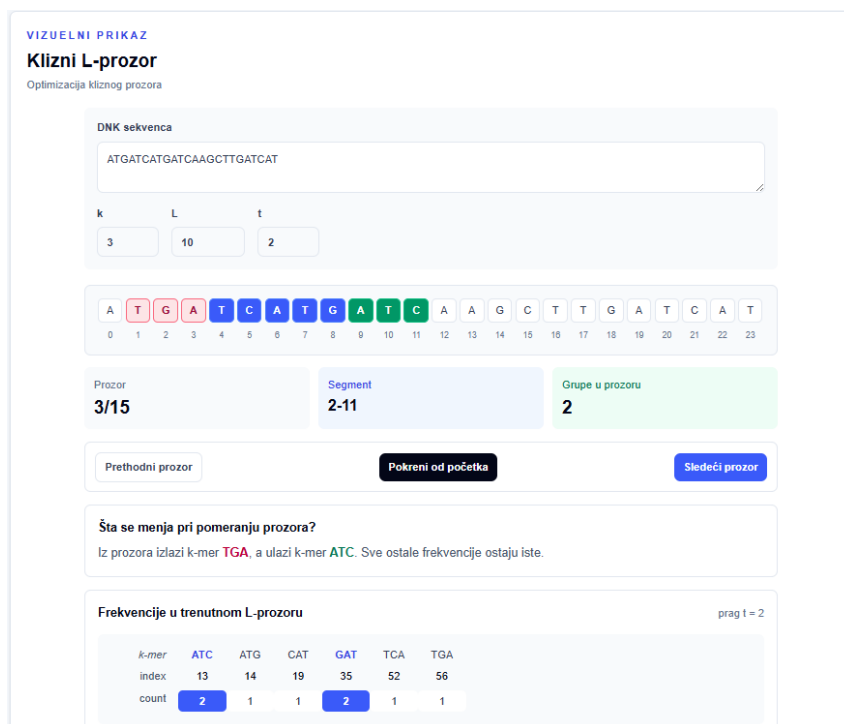
pozicija 2

Slika 4.7: Vizuelizacija naivnog algoritma za problem čestih reči

4.5.4 Pronalaženje grupa

Stranica Pronalaženje grupa prikazuje traženje k-grama koji formiraju (L, t) -grupe. K-gram formira grupu ukoliko postoji segment genoma dužine L u kojem se taj k-gram pojavljuje najmanje t puta.

Korisnik unosi DNK sekvencu i parametre k , L i t . Aplikacija prikazuje prozore dužine L , frekvencije k-grama u trenutnom prozoru i k-grame koji zadovoljavaju zadati prag. Pored osnovnog pristupa, prikazana je i optimizacija pomoću kliznog prozora, gde se pri svakom pomeranju prozora ažuriraju samo vrednosti za k-gram koji izlazi i k-gram koji ulazi u posmatrani segment. Vizuelizacija ovog pristupa prikazana je na Slici 4.8.

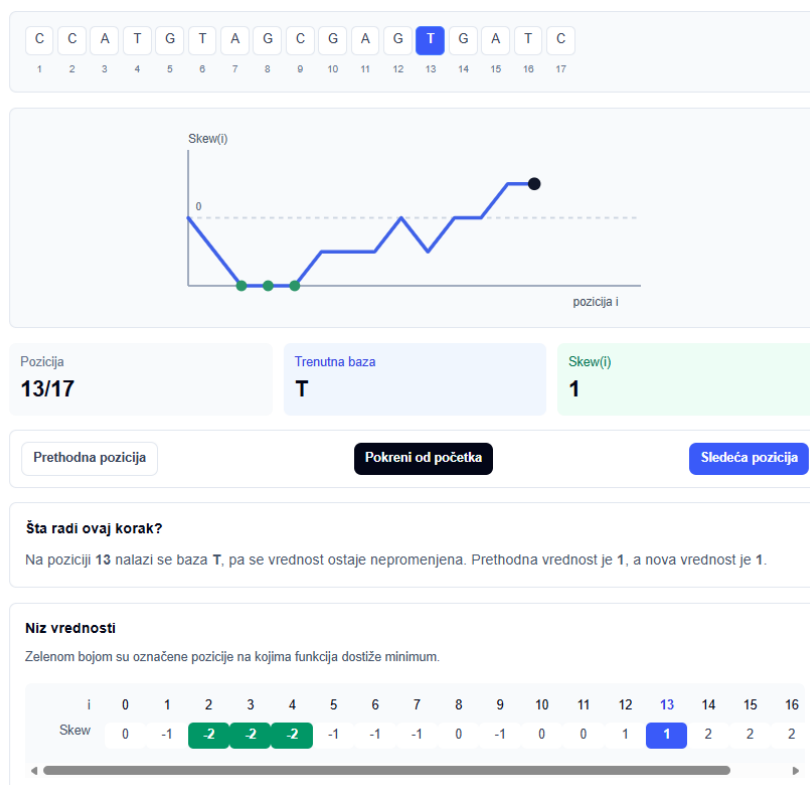


Slika 4.8: Vizuelizacija optimizovanog pristupa za pronalaženje grupa pomoću kliznog prozora

4.5.5 GC-skew dijagram

Stranica *GC-skew* dijagram prikazuje algoritam za izračunavanje *GC-skew* vrednosti duž DNK sekvence. Ova vrednost prati razliku između broja baza G i C u prefiksu sekvence. Pozicije na kojima *GC-skew* funkcija dostiže minimum mogu predstavljati kandidate za početni region replikacije.

Korisnik unosi DNK sekvencu, a aplikacija prikazuje konstrukciju niza vrednosti korak po korak. Vizuelni prikaz obuhvata trenutnu bazu, trenutnu vrednost funkcije, grafički prikaz *GC-skew* dijagrama i označene pozicije minimuma (Slika 4.9).



Slika 4.9: Vizuelizacija konstrukcije GC-skew dijagrama

4.5.6 Približno poklapanje i česte reči sa propustima

Stranica Propusti obuhvata više povezanih algoritamskih pojmova: Hamingovu distancu, približno poklapanje, d-susedstvo, česte reči sa propustima i obrnute komplemente. Ovi pojmovi su važni jer se biološki značajni motivi ne moraju uvek pojaviti u potpuno istom obliku. Mutacije mogu promeniti pojedinačne nukleotide, dok motiv i dalje može zadržati svoju funkciju.

Kao što je prikazano na Slici 4.10, za Hamingovu distancu korisnik unosi dve sekvence iste dužine, a aplikacija prikazuje broj pozicija na kojima se one razlikuju.

INTERAKTIVNI PRIKAZ

Hamingova distanca

Poredi dve sekvence iste dužine i broji pozicije na kojima se simboli razlikuju.

Prva sekvenca	AACCT					Druge sekvenca	AATCT				
Prva	A	A	C	C	T						
Druge	A	A	T	C	T						

Dužina

5

Različiti pozicija

1

Sličnost

4/5

Kako se računa?

Algoritam poredi simbole na istim pozicijama. Svaka kolona u kojoj se baze razlikuju povećava distancu za 1. Ovde je Hamingova distanca 1.

Slika 4.10: Interaktivni prikaz računanja Hamingove distance za dve sekvence

Kod približnog poklapanja korisnik unosi tekst, obrazac i dozvoljeni broj odstupanja, nakon čega se prikazuju pozicije na kojima se obrazac pojavljuje uz najviše d razlika. Deo za d -susedstvo prikazuje skup svih sekvenci koje su od zadatog obrasca udaljene najviše d pozicija (Slika 4.11).

INTERAKTIVNI PRIKAZ

d-susedstvo

Generiše sve sekvence koje se od obrasca razlikuju u najviše d pozicija.

Obrasc	AAAAA					d	1
Obrasc	AAAAA					Dozvoljena odstupanja	1
						Broj suseda	16

Susedstvo(obrazac, d)

Susedstvo sadrži sve sekvence iste dužine koje su udaljene najviše d pozicija od početnog obrasca.

AAAAA	AAAAC	AAAAG	AAAAT	AAACA	AAAGA	AAATA	AACAA	AAGAA	AATAA
ACAAA	AGAAA	ATAAA	CAAAA	GAAAA	TAAAA				

Slika 4.11: Prikaz d -susedstva niske AAAAA, za $d=1$

Na istoj stranici obrađuju se i česte reči sa propustima, gde se traže obrasci koji imaju najveći broj približnih pojavljivanja u tekstu, kao i efikasniji pristup pomoću heš mape. Dodatno, prikazana je i varijanta algoritma koja uzima u obzir obrnute komplemente, što je značajno pri analizi DNK sekvenci jer se motivi mogu posmatrati u obe orijentacije. Na Slici 4.12 vidimo interaktivni prikaz za česte reči sa propustima.

INTERAKTIVNI PRIKAZ

Česte reči sa propustima

Pronalazi k-mere koji imaju najveći broj približnih pojavljivanja u tekstu.

Tekst

CATAAATTCGTATGTATCAAATTTGTACTATCACATAAATTCGTATGTATCAAATTTGTACTATCA

k

6

d

1

Close kandidata

586

Max FrequencyArray

8

Najčešćih obrazaca

3

FrequencyArray za Close candidate

Prvo se susedi svih k-mera označe u nizu Close. Zatim se samo za te kandidate računa ApproximatePatternCount, a dobijene vrednosti se upisuju u FrequencyArray.

Obrazac	Broj
AAAATT	8
AAATTT	8
AATTTT	8
AAAAAT	6

Slika 4.12: Interaktivni prikaz algoritma za česte reči sa propustima

4.5.7 Literatura

Stranica Literatura sadrži izvore korišćene za izradu teorijskog i algoritamskog dela lekcije. Pored literature iz oblasti molekularne biologije i bioinformatike, na ovoj stranici nalazi se i link ka *GitHub* repozitorijumu u kojem se nalaze izvorni kod aplikacije, prateći projektni fajlovi i tekstualna verzija rada.

Glava 5

Zaključak

U ovom radu analizirani su algoritamski pristupi za identifikaciju početnog regiona replikacije u DNK sekvencama. Kroz prikaz različitih algoritama pokazano je na koji način se analiza učestalosti podniski, njihove lokalne raspodele i statističkih karakteristika genoma može iskoristiti za pronalaženje kandidata za *oriC* region.

Za svaki obrađeni algoritam analizirani su princip rada, složenost, kao i njegove prednosti i ograničenja. Prikazane optimizacije pokazale su kako se odgovarajućim izborom struktura podataka i algoritamskih tehnika može značajno unaprediti efikasnost obrade DNK sekvenci.

Praktični doprinos ovog rada predstavlja razvoj elektronske lekcije koja objedinjuje teorijska objašnjenja obrađenih algoritama i njihovu interaktivnu primenu kroz veb aplikaciju. Implementirana aplikacija omogućava korisnicima da pokreću algoritme nad proizvoljnim ulaznim podacima, prate njihovo izvršavanje korak po korak i analiziraju dobijene rezultate, čime se olakšava razumevanje njihovog rada i primene u bioinformatičari.

Iako opisani algoritmi predstavljaju značajnu osnovu za identifikaciju početnog regiona replikacije, u praksi se oni često kombinuju sa drugim bioinformatičkim metodama i biološkim informacijama kako bi se postigla veća pouzdanost rezultata. Budući razvoj može obuhvatiti proširenje elektronske lekcije novim algoritmima i interaktivnim primerima, kao i primenu opisanih metoda na većim skupovima realnih genomskih podataka, čime bi se dodatno unapredila njena obrazovna i praktična vrednost.

Bibliografija

- [1] Fastapi dokumentacija, 2026. on-line at: <https://fastapi.tiangolo.com/>.
- [2] Github repozitorijum sa izvornim kodom elektronske lekcije, 2026. on-line at: <https://github.com/dunjaci/master-rad>.
- [3] Next.js dokumentacija, 2026. on-line at: <https://nextjs.org/docs>.
- [4] Node.js dokumentacija, 2026. on-line at: <https://nodejs.org/en/docs>.
- [5] Python 3.12 dokumentacija, 2026. on-line at: <https://docs.python.org/3.12/index.html>.
- [6] React dokumentacija, 2026. on-line at: <https://react.dev/>.
- [7] Render dokumentacija, 2026. on-line at: <https://render.com/docs>.
- [8] Tailwind css dokumentacija, 2026. on-line at: <https://tailwindcss.com/docs>.
- [9] Vercel dokumentacija, 2026. on-line at: <https://vercel.com/docs>.
- [10] B. Alberts, A. Johnson, J. Lewis, M. Raff, K. Roberts, and P. Walter. *Molecular Biology of the Cell*. W. W. Norton & Company, 7th edition, 2022.
- [11] Vinod Chugani. Hamming Distance Explained: The Theory and Applications, 2025. on-line at: <https://www.datacamp.com/tutorial/hamming-distance>.
- [12] Jovana Kovačević. Materijal sa predavanja „Uvod u bioinformatiku”, 2022. on-line at: https://www.bioinformatika.matf.bg.ac.rs/predavanja/Chapter_1.pdf.
- [13] Adam R. Leman and Eishi Noguchi. The replication fork: Understanding the eukaryotic replication machinery and the challenges to genome duplication, 2013. Genes, 4(1), 1–32. on-line at: <https://doi.org/10.3390/genes4010001>.

- [14] Pavel Pevzner Philip Compeau. Bioinformatics Algorithms: An Active Learning Approach Vol. I, Chapter 1: Where in the Genome Does DNA Replication Begin?, 2015. on-line at: <https://www.bioinformaticsalgorithms.org/>.
- [15] Aleksandar Veljković. Uvod u bioinformatiku: Materijali sa časova vežbi - Pronalaženje početka replikacije, 2022. on-line at: <https://github.com/aleksandar-veljkovic/MATF-Uvod-u-bioinformatiku/tree/master/2022>.

Biografija autora

Dunja Čitlučanin rođena je 22. septembra 1998. godine u Beogradu. Smer Računarstvo i informatika na Matematičkom fakultetu Univerziteta u Beogradu upisala je 2017. godine, a završila 2022. godine. Nakon toga je upisala master studije na istom smeru. Od 2023. do novembra 2024. godine je zaposlena na poziciji *Junior Software engineer* u firmi **Telekom Srbija**, a potom od novembra 2024. godine do marta 2026. je zaposlena na poziciji *Software developer* u firmi **Development and Testing Center**. Sada je zaposlena kao *Data scientist* u firmi **NCR Atleos**.